



FACULTY OF INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING

Tarek Ali

**NEXT-GENERATION INTRUSION DETECTION
SYSTEMS WITH LLMS: REAL-TIME ANOMALY
DETECTION, EXPLAINABLE AI, AND
ADAPTIVE DATA GENERATION**

Master's Thesis
Degree Programme in Computer Science and Engineering
June 2024

Ali T. (2024) Next-Generation Intrusion Detection Systems with LLMs: Real-Time Anomaly Detection, Explainable AI, and Adaptive Data Generation.
University of Oulu, Degree Programme in Computer Science and Engineering, 65 p.

ABSTRACT

As the cybersecurity landscape continues to evolve, the need for effective real-time network anomaly detection mechanisms grows ever more critical. This study advances the traditional machine learning approaches in Intrusion Detection Systems (IDS) by exploring the diverse applications of Large Language Models (LLMs). Building on the Cybertrust framework [1], which guides the development of systems integrating Explainable, Actionable, and Interpretable Artificial Intelligence, this thesis extends the framework by focusing on three key aspects. First, the HuntGPT system integrates Explainable AI (XAI) techniques and LLMs to improve the confidence of cybersecurity operations teams in evaluating AI-generated alerts. HuntGPT is a specialized intrusion detection dashboard that combines a Random Forest classifier with XAI frameworks like SHAP and Lime. Paired with a GPT-3.5 Turbo conversational agent, HuntGPT presents threat insights in an easily understandable format, enhancing user comprehension and interaction. Second, we evaluate the functionality and effectiveness of fine-tuned LLMs for classifying network anomalies. This involves comparing their performance to traditional machine learning approaches in terms of cost, speed, and ease of deployment. This investigation aims to prove the effectiveness of minimally preprocessed LLMs as intuitive and efficient candidates for seamless integration into cybersecurity infrastructure. Finally, we introduce SecGPT, an interactive LLM agent designed to streamline the penetration testing process and generate new attack profiles in dynamic environments. Paired with a GAN architecture, SecGPT synthesizes comprehensive data to improve model retraining and enhance IDS detection capabilities. The study concludes with a comprehensive discussion of the results, highlighting the potential of LLMs in continuous data generation and their pivotal role in enhancing the explainability of AI solutions within intrusion detection systems. The findings contribute to advancing the understanding of LLMs in cybersecurity, paving the way for future research and practical implementations in live network anomaly detection scenarios.

Keywords: Intrusion Detection, Security, XAI, Chatbots, Explainable AI, Security Awareness, LLMs, Finetuning, ComputerNetworks, Ethical Hacking

TABLE OF CONTENTS

ABSTRACT	
TABLE OF CONTENTS	
FOREWORD	
LIST OF ABBREVIATIONS AND SYMBOLS	
1. INTRODUCTION.....	7
1.1. Objective of the Thesis	8
1.2. Structure of the Thesis	9
2. BACKGROUND.....	10
2.1. Problem Definition.....	10
2.2. Network Traffic Anomaly Detection.....	10
2.3. Explainable AI	12
2.4. Conversational Agents for Cybersecurity.....	13
2.5. Enhancing Network Security with Transformer Models.....	14
2.6. LLMs in Cybersecurity Testing.....	14
2.7. Optimizing IDS for Dynamic Networks.....	15
3. ARCHITECTURE	17
3.1. Data Extraction Module	18
3.2. DataFlow Pipeline	19
3.3. Anomaly Explanation Module	21
4. DATASET	22
4.1. Review of Available Datasets.....	22
4.2. Generation Setup	26
4.3. Attack Scenarios.....	27
4.4. Analysis	31
4.5. Shortcomings	34
5. EXPERIMENTS.....	35
5.1. Experiment 1: Integration of Explainable AI with LLMs.....	35
5.1.1. Anomaly Detection Model.....	36
5.1.2. Elasticsearch Data Schemas	36
5.1.3. Evaluation.....	37
5.2. Experiment 2: LLMs As Classifiers	41
5.2.1. Data Balancing.....	41
5.2.2. Models	42
5.2.3. Evaluation.....	44
5.3. Experiment 3: Autonomous Data Regeneration for Augmentation	47
5.3.1. Sec-GPT	48
5.3.2. Synthetic Data Generation	50
5.3.3. Evaluation.....	52
6. DISCUSSION	55
6.1. Results	55
6.2. Challenges.....	56
6.2.1. Data Volume in Network Traffic Capture	56
6.2.2. Lack of Reliable Public Datasets	56

6.2.3. Adapting to Dynamic and Evolving Cyber Threats	56
6.2.4. Computationally Intensive Models	57
6.3. Future Work	57
7. CONCLUSION	58
8. REFERENCES	59

FOREWORD

The journey of conducting this research has been both exciting and rewarding, marked by its share of challenges and triumphs. I want to sincerely thank Panos Kostakos and Susanna Pirttikangas for their continuous support and guidance throughout this thesis. Their help and encouragement have been invaluable. I also appreciate my colleagues at the Center for Ubiquitous Computing (UBICOMP) for being part of this experience. Furthermore, I wish to extend my profound gratitude to the European Commission for the H2020 IDUNN funding. Their support has been instrumental in making this research possible. Thank you for your invaluable contribution. Finally, I am deeply grateful to my family and friends. Their encouragement and support have been a source of strength throughout my academic journey.

Oulu, June 13th, 2024

Tarek Ali

LIST OF ABBREVIATIONS AND SYMBOLS

LLM	Large Language Model
AI	Artificial Intelligence
NIST	National Institute of Standards and Technology
IoC	Indicators of Compromise
CTH	Cyber Threat Hunting
XAI	Explainable Artificial Intelligence
NIDS	Network Intrusion Detection Systems
SMEs	Small and Medium-sized Enterprises
SOC	Security Operations Center
SIEM	Security Information and Event Management
VAE	unsupervised Variational Autoencoder
ML	Machine Learning
NLP	Natural Language Processing
NLU	Natural Language Understanding
GPT	Generative Pre-trained Transformers
VM	Virtual Machine
VPN	Virtual Private Network
DDoS	Distributed Denial-of-Service
FTP	File Transfer Protocol
SSH	Secure Shell Protocol
DBMS	Database Management System
SQL	Structured Query Language
DOS	Denial of Service
XSS	Cross-Site Scripting
C2	Command and Control
SMB	Server Message Block
HTTP	Hypertext Transfer Protocol
DVWA	Damn Vulnerable Web Application
GAN	Generative Adversarial Network
TGAN	Tabular GAN
CTGAN	Conditional Tabular GAN
RAG	Retrieval Augmented Generation
PSI	Population Stability Index
GPU	Graphics Processing Unit

1. INTRODUCTION

In recent decades, there has been a substantial escalation in cyber-attacks targeting critical and enterprise infrastructure. By 2025, anticipated annual financial damages from these cyber-attacks are projected to reach \$10.5 trillion USD, a substantial leap from \$3 trillion USD recorded in 2015 [2]. To counteract the evolving cyber threats, the National Institute of Standards and Technology (NIST) introduced a Cybersecurity Framework in 2014. This framework prescribes iterative cybersecurity policies for identification, protection, detection, response, and recovery processes related to cyber incidents [3].

In this backdrop, human experts play a vital role in analysing extensive telemetry data and Indicators of Compromise (IoC) to isolate real threats [4, 5]. Consequently, building on the foundation laid by the NIST Framework, an extensive ecosystem—comprising tools, methodologies, and techniques—has been established to enable the proactive identification of threats, a process referred to as Cyber Threat Hunting (CTH) [6, 7, 8]. Threat-hunting tools, such as Intrusion Detection Systems (IDS), enable analysts to apply their specialized knowledge to formulate and test threat hypotheses by analysing system telemetry as well as threat intelligence from external sources [4].

Machine learning-based anomaly detection tools are particularly noteworthy, designed to uncover both known and unknown threats. Generally, network anomalies are categorized into performance-related, such as file server failures and transient congestion, and security-related anomalies like Denial of Service attacks, spoofing, and network intrusions [9]. Evidently, the infusion of machine learning into cyber threat hunting tools has notably increased the incidence of false positives in real-world operational environments [10].

Explainable Artificial Intelligence (XAI) is at the focus of several proposed conceptual enhancements to existing cybersecurity frameworks, aiming to address the challenges brought forth by the integration of machine learning [11] in threat hunting and intrusion detection systems. A pivotal advancement in this realm is the evolution of *Cybertrust* frameworks, accentuating the necessity to integrate explainable, interpretable, and actionable AI in cybersecurity operations [1]. Nevertheless, the swift advancements in the domain may lead to information overload for incident responders and ML model maintainers, potentially resulting in a sluggish adoption rate [12].

Large Language Models, driving the rapid development of autonomous agents, show significant potential to transform the landscape of cybersecurity [13, 14, 15, 16]. Their ability to generalize across various downstream tasks by leveraging advanced pattern recognition, coupled with seamless integration into diverse AI tasks like classification, response assistance, explainability, and network testing automation, positions them as versatile solutions within cybersecurity operations.

This thesis addresses key challenges in integrating LLMs into IDS operations. It begins by generating an updated dataset essential for evaluating and benchmarking our proposed systems. Subsequently, we develop three distinct experiments: first, integrating explainable AI techniques with LLMs to enhance transparency; second, leveraging transformer-based models for anomaly classification; and third, generating synthetic data to adapt to dynamic network environments.

1.1. Objective of the Thesis

In this study, our objective is to integrate actionable, interpretable, and explainable AI into cybersecurity operations. We present a real-time pipeline designed for network anomaly detection, serving as our testbed for both dataset generation and evaluating the functionality of LLMs in our specific use case. The thesis addresses three pivotal research questions:

1. Can LLMs contribute to explainability and alleviate the blackbox nature of ML algorithms? Are they capable of providing substantial knowledge in the cybersecurity field for professionals?

To explore this, we commence with the development of HuntGPT, an interactive dashboard utilizing a Random Forest classifier as the core anomaly detection model. The model systematically classifies acquired packets, and to enhance interpretability, we integrate explainability frameworks like SHAP and LIME. Additionally, a conversational agent, powered by the Language Learning Model API from OpenAI, augments user interaction. The prototype undergoes a meticulous evaluation for technical accuracy using the Certified Information Security Manager Practice (CISM) certification [17], and it is assessed across six distinct metrics to gauge the quality of response readability.

2. Can LLMs outperform traditional ML in classifying anomalous traffic?

This investigation involves fine-tuning multiple pretrained language models as BERT and T5, on our carefully curated dataset of network traffic. By comparing their classification performance against traditional ML models, we aim to discern the effectiveness of LLMs in accurately identifying and classifying anomalies. The evaluation encompasses metrics like precision, recall, and F1 score to provide a comprehensive understanding of their capabilities.

3. How can LLMs address the challenge of increasing complexity and diverse network topologies in generating new data for retraining models?

To tackle the evolving complexity of network environments, LLMs are employed to generate novel data reflecting different network topologies. Leveraging their natural language processing capabilities, LLMs assist the user in automating the penetration process and creating multiple dynamic attack scenarios that capture the intricacies of modern networks. This novel approach aims to provide a solution to the perpetual challenge of adapting models to the dynamic nature of evolving network structures. The generated data is then synthesized for a more comprehensive final output and utilized for retraining models, ensuring their adaptability to a wide range of network configurations.

1.2. Structure of the Thesis

- **Chapter 2** explores relevant literature, summarizing key findings from studies related to Network Traffic Anomaly Detection, Explainable AI, Conversational Agents for Cybersecurity, Enhancing Network Security with Transformer Models, LLMs in Cybersecurity Testing, and Optimizing IDS for Dynamic Networks. This in-depth review sets the stage for understanding the current research landscape.
- **Chapter 3** describes the testbed system's architecture and implementation details. This chapter provides insights into the technical aspects and design choices that form the backbone of our research.
- **Chapter 4** discusses the creation process of our dataset, explaining the methods we used to tailor it to our research objectives.
- **Chapter 5** offers detailed insights into three experiments conducted on the proposed testbed architecture. It examines the results of each experiment, focusing on LLMs and their integration with Explainable Artificial Intelligence, their use as classifiers, and their role in data generation.
- **Chapter 6** discusses the results, challenges, and suggests directions for future research.
- **Chapter 7** offers summaries and concludes the output of the thesis.

2. BACKGROUND

2.1. Problem Definition

Adopting efficient cybersecurity strategies is challenging for small and medium-sized enterprises (SMEs) due to constraints such as limited budgets, a lack of skilled personnel, and insufficient time allocated to cybersecurity planning [18]. Table 1 illustrates the typical expenses, totalling \$1,635,000, required to sustain a medium-sized Security Operations Center (SOC) team, accounting for both personnel and infrastructure. Effectively, this cost analysis sheds light on the challenges faced by smaller organizations and emphasizes the importance of accessible and cost-efficient cybersecurity solutions.

Table 1. An estimate of the cost of a medium size SOC team [19]

Role	Count	Yearly Cost	Total Cost
Tier 1 Analyst	5	80,000 \$	400,000 \$
Tier 2 Analyst	3	100,000 \$	300,000 \$
Tier 3 Analyst	2	120,000 \$	240,000 \$
Malware Engineer	1	120,000 \$	120,000 \$
Forensic specialist	1	130,000 \$	130,000 \$
SOC manager	1	140,000 \$	140,000 \$
Skill development	13	5,000 \$	65,000 \$
Products/Infrastructure*	1	240,000 \$	240,000 \$
Total cost for People / Infrastructure			1,635,000 \$

* SIEM, EDR, Server Hardware, Forensic Software, and VAPT Tools

Cyber defense is a crucial domain within cybersecurity focused on detecting, preventing, and responding to cyber threats. The primary goals of cyber defense are to protect digital systems from threats and to develop resilient systems capable of recovering from attacks. The increasing sophistication of cyber attacks necessitates the development of proactive defense technologies, urging the research community to invest significant effort in staying ahead of evolving attack techniques [20]. In this section, we explore advancements in several key areas that are essential to understand the work we present in the following sections, including Network Traffic Anomaly Detection, Explainable AI, Conversational Agents for Cybersecurity, Enhancing Network Security with Transformer Models, Actionable, LLMs in Cybersecurity Testing and Data Regeneration for Improved IDS.

2.2. Network Traffic Anomaly Detection

The purpose of an anomaly detection mechanism is to analyze, understand and characterize network traffic behavior, as well as to identify or classify the abnormal traffic instances such as malicious attempts from normal instances. Thus, from a machine learning perspective, the anomaly detection problem is a classification

problem [21]. Over the years, detection systems have experienced considerable evolution, resulting in the development of diverse approaches and deployment methods, including those in fifth-generation (5G) communication networks and decentralised architectures [13, 22, 23, 24].

Several Machine Learning (ML) techniques have been extensively applied in the domain of network anomaly detection, encompassing both supervised and unsupervised algorithms. Yihunie et al. [25] reviewed five representative algorithms: Stochastic Gradient Descent, Random Forests, Logistic Regression, Support Vector Machine, and Sequential Model, applying them to the NSL-KDD dataset. The empirical results from their study indicated that the Random Forest Classifier surpassed the other examined classifiers in terms of performance.

Eltanbouly et al. [26] introducing a hybrid system that combines the Random Forest and K-means algorithms. The proposed system is bifurcated into two distinct phases. The initial phase, known as the online phase, focuses on misuse detection by leveraging the Random Forest algorithm, followed by the offline phase, which categorizes random attacks through the use of the weighted K-means algorithm. Similarly, Zhao et al. [27] proposed a Multi-Task Deep Neural Network in Federated Learning (MT-DNN-FL) to simultaneously detect network anomalies, recognize VPN (Tor) traffic, and classify traffic, while preserving data confidentiality. Experimental results on representative datasets demonstrated superior performance in detection and classification compared to several baseline methods in centralized training architecture.

Preuveneers et al. [28] proposed a blockchain-based federated learning method, allowing for the auditing of model updates without centralizing training data, thus providing enhanced transparency and accountability in detecting malicious behavior. The experiments show that while integrating blockchain increases complexity, the impact on performance is minimal (varying between 5 and 15%), and the method is adaptable to more sophisticated neural network architectures and diverse use cases. Noticeably, the adoption of federated self-learning for anomaly detection and threat hunting is a rising trend in IoT devices, focusing on enhancing detection accuracy while prioritizing data privacy [23, 29, 30].

Lately, the widespread adoption of 5G networks has increased the demand for the development of automated Intrusion Detection Systems, leading to a boost in specialized research in this domain [31]. Sheikhi et al. [24] focused on employing a federated learning-based method to identify DDoS attacks on the GTP protocol within a 5G core network. This approach capitalizes on the collective intelligence of various devices to proficiently and confidentially recognize DDoS attacks. While ML models exhibit strong performance in controlled settings, their efficacy in real-world environments is perceived as a significant barrier to their adoption [10]. In the following section, we will discuss various explainability methods that have been proposed as potential solutions to overcome the barriers associated with the real-world performance of ML models.

2.3. Explainable AI

Explainable artificial intelligence (XAI) refers to advanced techniques that aim to make the results of machine learning models understandable and more transparent to users. These techniques allow a practical deployment of ML models as they provide methods to ensure that the trained model is trustworthy by detecting biases in the model or in the corresponding training data, and increasing transparency in the predictions of the models by providing explanations for which input features had the most impact in the output of the model. For many critical applications in defense, medicine, finance, and law, explanations are essential for users to understand, trust, and effectively manage these new, artificially intelligent partners [32]. Recent research in Explainable Artificial Intelligence (XAI) has been actively applied to cybersecurity, particularly in specialized use cases like intrusion detection and malware identification[1].

Nguyen et al. develop GEE [33], a framework for detecting and explaining anomalies in network flow traffic. GEE comprises of two components. The first one consists of an unsupervised Variational Autoencoder (VAE) model for detecting network anomalies. The second one is a gradient-based fingerprinting technique for explaining the detected anomalies in the VAE. The evaluation shows that their approach is effective in detecting different anomalies as well as identifying fingerprints that are good representations of these attacks.

Han et al. developed the DeepAID framework [34] to interpret unsupervised DL-based anomaly detection systems for cybersecurity. The approach helps security analysts understand why a certain sample is considered anomalous by searching the difference between the anomaly and a normal reference data point. Additionally, they propose a model distiller that serves as an extension to the black-box DL model using a simpler and easier to understand finite-state machine model that allows analysts to get involved in the model decision-making process. While XAI holds promise in enhancing the adoption of ML models within existing cybersecurity frameworks, there are still several challenges and considerations to be address.

Nyre-Yu et al. [12] conducted a pilot study within an operational environment to assess an Explainable Artificial Intelligence (XAI) tool, focusing on insights gleaned from real-time interactions between cybersecurity analysts and XAI. The initial findings disclosed that, contrary to the goal of fostering trust and improving efficiency through XAI tools, their actual deployment was restricted and failed to considerably improve the accuracy of the decisions made by the analysts. Similarly, a recent systematic review [35] highlighted that research on XAI for cybersecurity shows that many XAI applications are crafted without a thorough understanding of their integration into analyst workflows. Moreover, security literature frequently fails to differentiate between diverse use cases or clearly separate the roles of model users and designers, which could potentially lead to diminished adoption.

2.4. Conversational Agents for Cybersecurity

Conversational agents, often referred to as chatbots, have gained attention for their role in supporting cybersecurity within businesses through sharing network and security information with non-technical staff [36].

The security-focused chatbot introduced in [37] named SecBot demonstrates the role of a conversational agent for the support of cybersecurity planning and management. SecBot applies concepts of neural networks and Natural Language Processing (NLP) to interact and extract information from a conversation to (a) identify cyber-attacks, (b) indicate solutions and configurations, and (c) provide insightful information for the decision on cybersecurity investments and risks.

Another notable advancement in Natural Language Understanding (NLU) with proven success in cybersecurity is the development of Generative Pre-trained Transformers (GPT) language models.

These models can operate as standalone tools; for example, the application of GPT in formulating cybersecurity policies, as demonstrated in McIntosh et al. [38], helps deter and mitigate the impact of ransomware attacks involving data exfiltration. The results of the study indicated that, in specific scenarios, policies generated by GPT could surpass those created by humans, especially when supplied with customized input prompts. Similarly, Setianto et al. [39] developed a run-time system, GPT-2C, that utilizes a fine-tuned GPT-2 model to parse logs from a live Cowrie SSH honeypot effectively, achieving 89% inference accuracy in parsing Unix commands with minimal execution latency.

Furthermore, research emphasis has been placed on exploring the potential threats posed by Large Language Models (LLMs) like OpenAI's ChatGPT and Bard, particularly regarding their abilities to facilitate cyberattacks [40, 41]. Nonetheless, limited research has been conducted on the potential integration of LLMs into cyber-hunting interfaces. Bringing together conversational agents and GPT offers new opportunities for delivering knowledge and insights to non-professionals. Preliminary research and trials from other domains provide insight into this potential. For instance, [42] examines the viability of leveraging Explainable AI (XAI) and language models like ChatGPT to transform how financial knowledge is conveyed to those outside the financial sector. The findings suggest that ChatGPT holds significant promise in demystifying intricate financial principles for a broader audience.

Finally, prompt engineering [43], an emerging research area, is showing promise as a potential solution for some of the shortcomings of LLMs since simple and effective prompts have been proven to improve GPT-3's reliability. With appropriate prompts, GPT-3 is more reliable than smaller-scale supervised models on all these facets: generalizability, social biases, calibration, and factuality [44]. However, evaluating the performance of Large Language Models across diverse domains presents an ongoing challenge. Several research studies have examined ChatGPT's performance in diverse fields. Gilson et al. [45] investigated its performance on the United States Medical Licensing Examination, highlighting the ability of ChatGPT to provide logical and informative context in the majority of its responses. Notably, when ChatGPT delivers accurate and up-to-date information, users report higher satisfaction levels and make

more informed decisions [46]. Therefore, understanding and quantifying the accuracy of the data provided in the cybersecurity domain is of high importance.

2.5. Enhancing Network Security with Transformer Models

Gniewkowski et al. [47] explored the application of NLP techniques in detecting anomalies in HTTP traffic and identifying malicious URLs. The study compares different vectorization methods, including bag-of-words, fastText, and RoBERTa models to represent the data for classification tasks. By leveraging these NLP methods, the study aims to enhance the understanding of patterns within the data and improve classification accuracy. The researchers highlight the importance of being able to explain how the models arrive at their decisions, especially in the context of cybersecurity. By using techniques like the SHapley Additive exPlanations, the study aims to shed light on the features that contribute to the classification outcomes, making the results more interpretable. Moreover, the research addresses the concept drift phenomenon, which refers to the changes in the data distribution over time. By testing the robustness of their models on different datasets and scenarios, the study evaluates how well the proposed solutions can adapt to evolving environments. By combining innovative vectorization methods with explainability tools and considering concept drift challenges, the research contributes to the development of robust and interpretable cybersecurity solutions, highlighting the potential of NLP methods in enhancing security mechanisms and understanding complex patterns within security data.

This paper [48], presents a Deep Learning based Intrusion Detection System using BERT and GPT-2 algorithms for identifying and mitigating security threats in wireless communication networks. The system is evaluated using the CICIDS2017 intrusion dataset and shows promising results in terms of prediction accuracy and computational complexity. The study concludes with recommendations for future research to improve the model's performance, particularly in multiclass classification scenarios. Future research directions include expanding the feature set and improving the model's performance for multiclass classification scenarios.

2.6. LLMs in Cybersecurity Testing

Human expertise in penetration testing, a key measure for system security, has always been essential. Yet, with the progress in LLMs, there is a potential shift, indicating these models could revolutionize the industry.

This study [49] explores the use of LLMs in penetration testing through the development of a tool, PENTESTGPT. Comprising of three modules - Reasoning, Generation, and Parsing, each plays a key role in creating an efficient system for penetration tests. The Reasoning Module works as a team lead, setting the next steps the Generation Module then executes, while the Parsing Module streamlines user interaction. PENTESTGPT addresses LLM challenges like context loss and inaccurate result generation by using a pentesting task tree and a strategy that divides

the penetration testing into two processes. The research also outlines LLM limitations and acknowledges the importance of including human expertise for guidance.

This paper [50] investigates the potential of LLMs for automating cyber campaigns and analysing threats. The paper presents an automated threat campaign procedure using LLMs that simulates various stages of a cyberattack. However, it points out the limitations of the current LLM design (GPT-3.5 specifically), which relies on understanding command line outputs and generating executable commands, limiting its scope to the data it was trained on. The authors argue that while LLMs can be a powerful tool in cybersecurity if used responsibly, there are ethical considerations due to LLM's potential for misuse.

This paper [51] presents a study on the potential abilities of LLMs and their capability to autonomously hack websites. The research focuses on the interaction between AI technology models and cybersecurity. The presented findings reveal an impressive ability of these agents, specifically GPT-4, to detect and exploit vulnerabilities in created sandbox websites and in live websites, proving capable of intricate maneuvers like blind database schema extraction and SQL injections without any prior knowledge of the vulnerabilities.

2.7. Optimizing IDS for Dynamic Networks

Intrusion Detection Systems in today's networking environments face multiple challenges largely due to the growing complexity of these networks. The constant updates in service protocols and changes in how organizations use these services add to these challenges. Traditional IDSs, which often depend on static rules, known patterns and defined thresholds, struggle to maintain efficacy under these dynamic conditions. To overcome these issues, it's important to create adaptive security solutions that can perform real-time analysis and learning. These adaptive IDSs would be able to monitor and understand changes within the network, making it easier to tell the difference between normal activities and malicious threats. This not only improves the accuracy of threat detection but also reduces the number of false alarms, making the security management process more efficient in increasingly complex network settings.

Neelakantan and Rao proposed a novel threat-aware IDS model that is capable of learning changing threat exposures in its operating environment and can adapt itself for effective intrusion detection in [52]. The proposed architecture consists of three key components: the Threat Collector, Threat Profiler, and Detection Engines.

- **Threat Collector:** Gathers information through active vulnerability assessments and passive network traffic sniffing to generate threat profiles.
- **Threat Profiler:** Utilizes these profiles to create a signature set for signature-based detection and anomaly blueprints for anomaly detection. This includes modules like the Signature Selector and Anomaly Blueprint Generator.

- **Detection Engines:** Equipped for both signature and anomaly detection. They use the threat profiles to maintain relevant signatures and set dynamic thresholds based on threat levels, enhancing detection accuracy and reducing false positives.

The threat-aware IDS allows for precise alarm generation and effective anomaly detection. This approach has been validated with both isolated and live datasets, proving its effectiveness in dynamic network environments.

This paper [53] provides a game-theoretic solution to the IDS configuration problem in dynamic network environments, determining optimal detection thresholds for anomaly-based detectors. The problem is modeled as a security game between a defender and an attacker. The defender aims to set detection thresholds that minimize detection delay while maintaining acceptable false positive rates, while the attacker seeks to maximize damage by timing the attack optimally. The paper presents two strategies: a fixed threshold strategy and an adaptive threshold strategy. The fixed threshold strategy involves an algorithm to compute a time-independent threshold for the detector. To find the optimal detection delay, the algorithm iterates through all possible values of detection delay and selects the one that minimizes the defender's loss considering a best-response attack. The adaptive threshold strategy uses a polynomial-time algorithm to compute thresholds that vary over time, accounting for the changing potential damage from attacks. These methods are evaluated using a water distribution network as a case study. Results show that adaptive thresholds better balance detection delay and false positives, reducing defender losses more effectively than fixed thresholds in dynamic environments.

The authors of [54], address the challenge of false positives in Intrusion Detection Systems, particularly in dynamic network environments. The CLARAty method uses data mining to cluster historical alerts, identifying and addressing root causes, which reduces alerts by 75% on average. The ALAC approach employs machine learning, leveraging analyst feedback to classify alerts as true or false positives. Operating in recommender and agent modes, ALAC reduces the analyst's alert review workload by over 50% while maintaining high classification accuracy. Both methods can significantly improve IDS alert management when integrated into standard systems.

3. ARCHITECTURE

Prior to undertaking our experiments, it is essential to establish a robust testbed with a comprehensive architecture. This foundation is critical for capturing the requisite data needed for model training and real-time testing. Illustrated in Figure.1, our architecture comprises three interconnected modular layers, each tailored to fulfill distinct functions and ensure optimal performance. Below, we outline the elements and functions of each layer, clarifying their roles and interactions within the overall system and their significance in facilitating our experiments.

1. **Data Extraction Module:** The initial module is straightforward, relying on the CICFlowMeter [55] developed by the Canadian Institute of Cybersecurity. However, we have made our own enhancements to the tool. Originally designed to convert packet data into meaningful records and store them locally for dataset generation, we have modified the tool's source code to enable data transmission to our DataFlow pipeline.
2. **DataFlow Pipeline:** Upon receiving network records at the pipeline entry point API, the initial step involves storing them in a Kafka broker for permanent storage. These records are retained for subsequent use in experiments, dataset generation, and model training. Subsequently, the data proceeds through a series of processing stages, including imputation, scaling, renaming, and the removal of less valuable columns. Finally, the records undergo classification, determining whether they are anomalous and, if so, the type of anomaly they represent.
3. **Anomaly Explanation Module:** The third module is called for whenever an anomaly is detected. This comprehensive module comprises multiple submodules, each of which will be thoroughly explored in the following sections. At its core lies an analytics engine responsible for generating all anomaly data, connected to explainability frameworks. Additionally, a data storage submodule is integral, harnessing Elasticsearch as our primary document storage. Esteemed for its near real-time search capabilities, scalability, and reliability, Elasticsearch houses all detected network anomalies alongside the corresponding original flow data. For the storage of plots and images, we rely on Amazon S3 buckets, ensuring both security and accessibility. Finally, we have built a user interface using the Gradio Python package, serving as the interactive front-end of our system. This interface presents the outcomes of the analytics engine to human analysts in a user-friendly manner. Additionally, it is integrated with OpenAI's API, facilitating seamless interactions between analysts and the system for ongoing discussions and analysis.

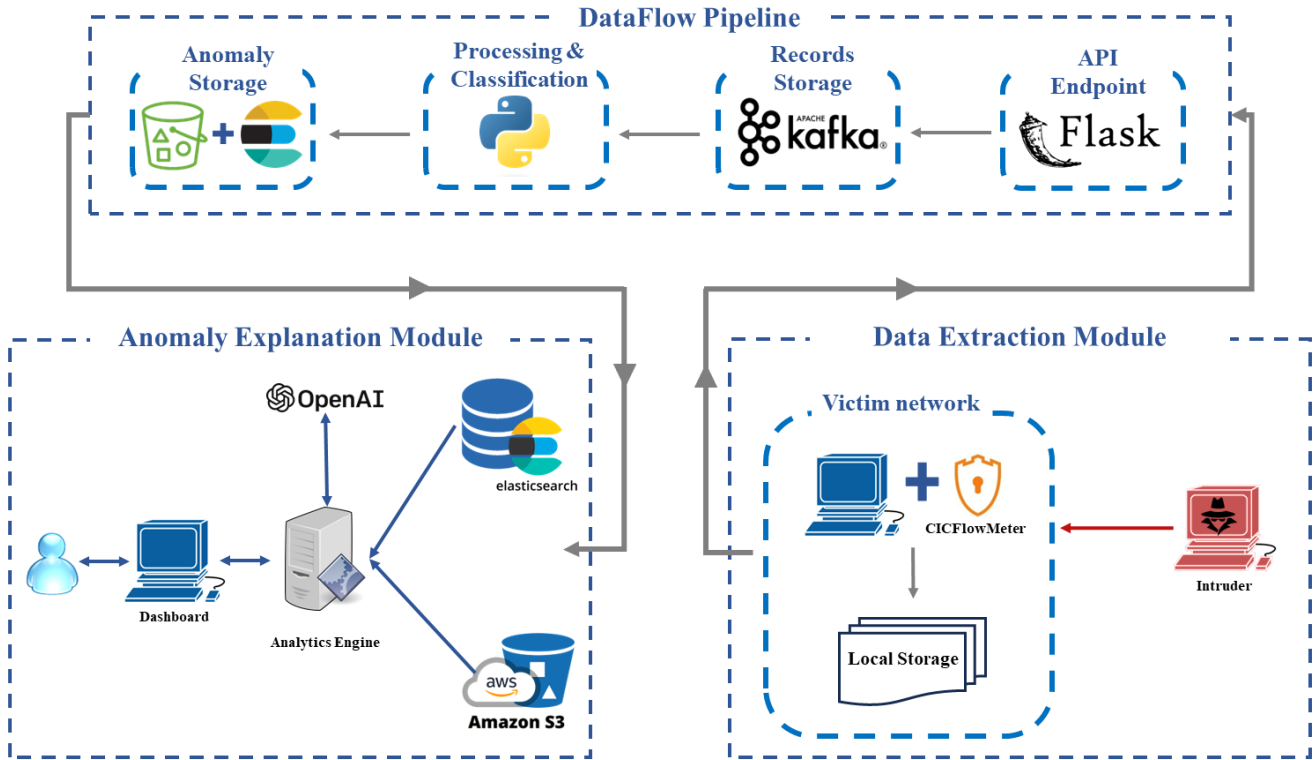


Figure 1. High-level diagram of Testbed Architecture.

We chose this configuration due to its modular design, placing a premium on the segregation of components allowing each layer to be developed, maintained, and enhanced independently. This configuration enables us to conduct numerous experiments, both locally using the generated dataset and in real-time scenarios. The autonomous nature of each module ensures seamless adaptability to changing requirements, enabling further enhancements and scalability for the network in the future. In the following sections, we narrow our focus to provide more detailed insights into the architecture of our system, engaging in a concise discussion on each primary component.

3.1. Data Extraction Module

Our data extraction system follows a straightforward approach. We set up an attack and a victim machine within a closed, controlled environment to prevent data contamination. The victim machine, powered by Linux Ubuntu 22.04, hosts a tailored CICFlowMeter. Meanwhile, the attack machine operates on Kali Linux 2023.2, enabling the execution of various scripts and commands to simulate different network attacks.

Figure 2 depicts a comprehensive illustration of our network diverse components. The DataFlow pipeline receives all attack records captured on the victim machine for additional preprocessing. This approach alleviates the need for high computational power on the victim machine, thereby expanding the potential for additional applications.

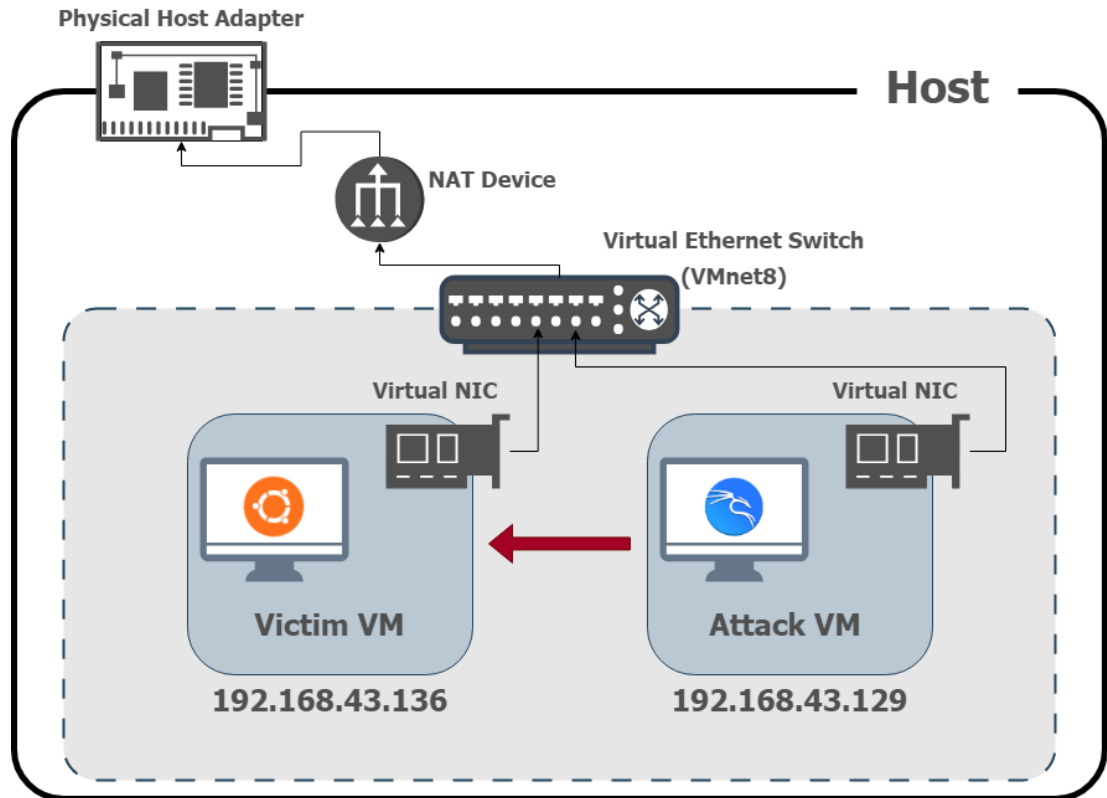


Figure 2. Network Architecture

3.2. DataFlow Pipeline

Our pipeline operates in three main stages as shown in Figure 3, each crucial for data management and processing. Firstly, we use Virtual Machines (VMs) from CSC, a Finnish IT Center for Science, to receive and store data securely to deploy our tools. The Data Ingestion step starts by efficient data collection using a Flask API endpoint. While currently, we are testing with just one VM to keep costs down, our system can easily scale up by containerizing and deploying it across major cloud platforms. For our storage and retrieval process, we rely on Kafka, which provides a distributed, fault-tolerant, and highly scalable storage and retrieval service for real-time data streams. Kafka allows data ingestion, persistence, and querying in a fault-tolerant manner. The second step, Data Processing and Classification involves classifying the network flow record as normal or anomalous. When an anomaly is detected, the record is passed to the anomaly module for further processing, providing the user with an explanation of the alert.

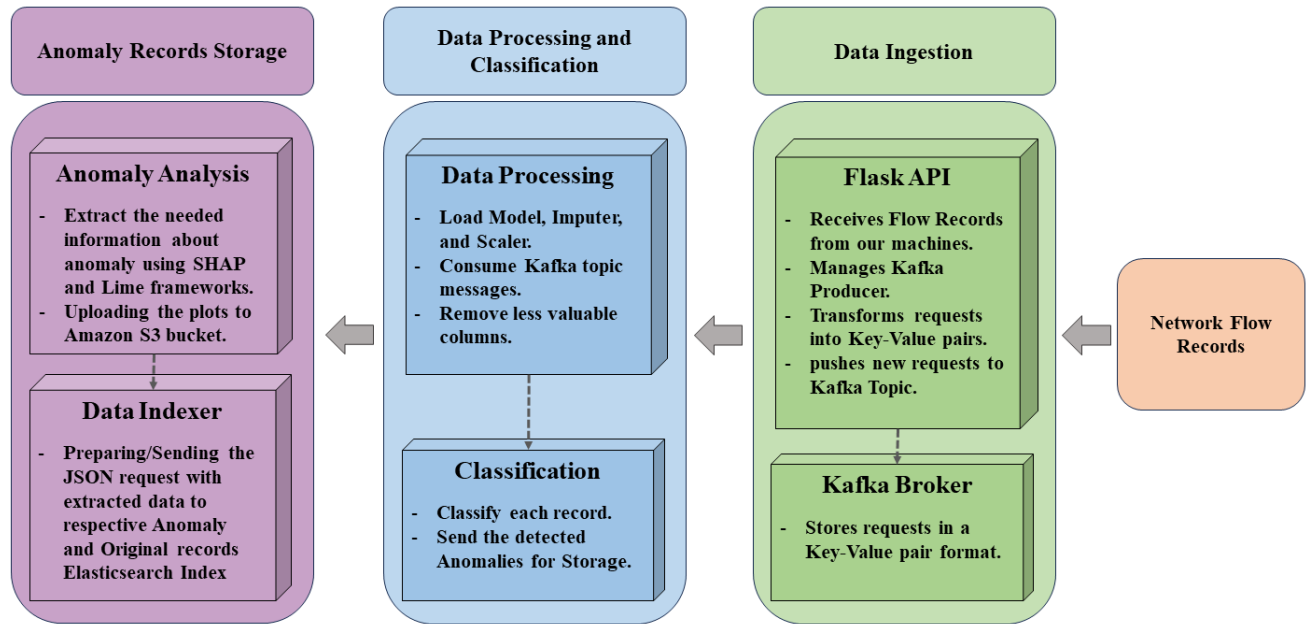


Figure 3. DataFlow Pipeline Diagram

The DataFlow Pipeline Server serves as the central orchestrator of the entire anomaly detection process. This component integrates several sub-modules to facilitate the efficient and accurate identification of anomalous network behavior.

1. **ML Model Loader:** Loading the pre-trained machine learning model to assess incoming data points and then process the anomalies through explainability frameworks (SHAP and LIME) to provide interpretable explanations for predictions.
2. **Elasticsearch Connector:** Enabling seamless communication with Elasticsearch, the connector module handles authentication and index management, ensuring secure access to the Elasticsearch cluster. This components handles establishing a secure connection, creating and verifying indices, eventually enabling efficient storage of information pertaining to detected and original packets.
3. **Prediction:** The prediction component analyzes individual network flows to determine the presence of anomalies.
4. **Explainer:** This component generates prediction factors, plots, and JSON documents. The explainer indexes the generated data into Elasticsearch, constructing a structured repository that facilitates efficient querying and exploration of detected and original network flows. To augment the interpretability of our findings, the explainer component uploads plot images to an AWS S3 bucket. These plots enhance our understanding of the model dynamics.
5. **Elasticsearch:** Elasticsearch plays a pivotal role in storing and organizing information. Our system leverages Elasticsearch to manage both the "Detected Packets Index" and the "Original Packets Index," optimizing data accessibility and analysis.

6. **AWS S3 Bucket:** Serving as a centralized repository for our visual resources housing the uploaded plots.

3.3. Anomaly Explanation Module

We have developed a specialized intrusion detection dashboard, termed HuntGPT, which implements a Random Forest classifier trained on our custom dataset. HuntGPT integrates Explainable AI (XAI) frameworks such as SHAP and LIME, enhancing the model's user-friendliness and intuitiveness. By combining these capabilities with a GPT-3.5 Turbo conversational agent, we aim to present detected threats in an easily understandable format, prioritizing user comprehension and providing a seamless interactive experience. A more detailed explanation of the architecture and inner workings will be presented in Section 5.1.

4. DATASET

Creating successful AI solutions depends largely on the quality of the data use, and this principle applies equally to the development of Network Intrusion Detection Systems (NIDS). Anomaly-based detectors attempt to estimate the “normal” behaviour of the system to be protected and generate an anomaly alarm whenever the deviation between a given observation at an instant and the normal behaviour exceeds a predefined threshold. Another possibility is to model the “abnormal” behaviour of the system and to raise an alarm when the difference between the observed behaviour and the expected one falls below a given limit [56]. Over the years, various datasets have been compiled specifically to train, test, and refine different methods for detecting unusual network activity.

As network behaviors and patterns change and intrusions evolve, it has very much become necessary to move away from static and one-time datasets toward more dynamically generated datasets which not only reflect the traffic compositions and intrusions of that time, but are also modifiable, extensible, and reproducible [57]. The process for generating datasets to train the classifier involves monitoring network traffic over a specific time interval. These datasets include both normal and anomalous network traffic, aiding the classifier in identifying data patterns with a sufficient amount of examples [58].

As argued by sommer and paxson [59] that arguably the most significant challenge an evaluation faces is the lack of appropriate public datasets for assessing anomaly detection systems. To be able to conduct our experiments and evaluate our methods successfully, we created a comprehensive dataset that reflects the complexity of today’s diverse cyberspace.

In this section, we will begin by examining the existing datasets 4.1, followed by an exploration of our dataset creation process 4.2, and then a comprehensive analysis of the dataset 4.4. Subsequently, we will discuss the limitations of current datasets and compare them with our newly generated one 4.5.

4.1. Review of Available Datasets

Developing comprehensive datasets for Intrusion Detection Systems is an ongoing challenge and even with the presence of multiple public datasets available, new work is being developed constantly because the current existing solutions frequently exhibit shortcomings, as outlined by Kenyon et al. [60]:

- **Poor provenance:** unclear methodology on how the dataset was created
- **Unclear composition:** with respect to benign and malicious event frequencies and scope
- **Poor data quality:** such as duplicate data, unrepresentative samples
- **Over-summarisation:** with potential loss of features, and inability to scrutinize origin data

- **Unmaintained ‘frozen’ data:** leading to unrepresentative threat event signatures
- **Limited to specific domains:** unrepresentative of commercial deployments (e.g. academic networks)
- **Under-represented domains:** such as industrial networks, IoT networks

We will now provide a short but comprehensive analysis for each of the most common publicly available IDS datasets since 1998 followed by the comparison demonstrated in Table 2 which illustrates the advantages and shortcomings reflecting the need for an updated reliable dataset.

DARPA: Sponsored by DARPA and AFRL, the MIT Lincoln Laboratory pioneered the creation of standard datasets for evaluating Intrusion Detection Systems (IDS). These datasets, crucial for training and testing IDS, have significantly influenced intrusion detection research. They encompass network traffic, audit data, and file system snapshots to identify intrusions. The DARPA IDS evaluation dataset includes labeled attacks categorized into Probe, DoS, R2L, U2R, and Data attacks, facilitating research in intrusion detection. The primary criticism of the DARPA IDS evaluation dataset is the lack of publicly available test bed traffic generation software, making it difficult to assess the accuracy of the background traffic. Additionally, the evaluation criteria do not consider factors such as system resources, ease of use, or system type [61].

KDD99: The dataset is a collection of simulated raw TCP dump data over a period of nine weeks on a local area Network. The training data was processed to about five million connections records from seven weeks of network traffic and two weeks of testing data yielded around two million connection records [62]. The dataset is built based on the data captured in DARPA’98 IDS evaluation program and inherits some of the criticized aspects of the DARPA’98 data, such as its synthetic nature and lack of validation for false alarm characteristics. Workload disparities and potential packet drops by traffic collectors like TCPdump further add to its challenges. Additionally, the dataset itself suffers from reliability issues, including redundant records that bias learning algorithms and evaluation outcomes. Anomaly detection is also challenging, often requiring custom machine learning algorithms trained on segmented data for accurate evaluation [63].

DEFCON: The DEFCON-10 dataset, captured in 2002 during a capture flag competition, includes port scans, sweeps, bad packets, administrative privilege, and FTP by Telnet protocol attacks. Primarily employed for assessing alert correlation techniques [64].

CAIDA: This dataset contains anonymized traffic traces from a DDoS attack on August 4, 2007, totaling approximately one hour. Split into 5-minute pcap files, it occupies 5.3 GB when compressed (21 GB uncompressed). It exclusively features attack traffic directed at the victim and its responses, with non-attack traffic minimized. Anonymization is achieved using CryptoPAN prefix-preserving anonymization, and

packet payloads have been removed [65].

LBNL: The aim of this project is to analyze internal enterprise traffic from a medium-sized site to understand similarities and differences compared to wide-area Internet traffic. Packet traces spanning over 100 hours of activity were collected from several thousand internal hosts. The dataset, comprising 11GB of packet header traces from October 2004 through January 2005, has been publicly released in anonymized form to protect individual IP identification [66].

CDX: This dataset was created to explore the use of network warfare competitions for generating labeled datasets. During a four-day Cyber Defense Exercise, network sensors strategically placed near data sources automatically characterized much of the network traffic. The competition, involving the U.S. Military Academy and an NSA red team, took place over a VPN with restricted external Internet traffic. Attackers automated reconnaissance and attacks using tools like Nessus, WebScarab, and Nikto, while organizers manually generated white traffic by manually interacting with web services, email, DNS lookups, and other essential services. Though useful for testing IDS alert rules, the dataset lacks diversity and volume in traffic [67].

Kyoto: An evaluation dataset constructed using three years of real traffic data (November 2006 to August 2009), sourced from various types of honeypots. It consists of 24 statistical features, including 14 conventional ones from the KDD Cup 99 dataset and 10 additional features to enhance the analysis of network activities such as IDS_detection, Malware_detection, and Ashula_detection. Given that most honeypot traffic data consist of attack data, normal traffic data is necessary to create a comprehensive IDS dataset. To generate normal traffic data, a server was deployed on the same network as the honeypots. The server offered mailing and DNS services for one domain and supported protocols like SSH, HTTP, and HTTPS for management. All traffic from this server was categorized as normal data for evaluation [68].

Twente: Represents the first labeled flow-based data set containing only flows while full-packet content was used only as additional information source during the labeling process. the data is gathered through a honeypot, hosted on a virtual machine using Citrix XenServer 5, ran Debian Etch 4.0 with services like OpenSSH, Apache, and proftpd for monitoring. Over a 6-day period, it logged all incoming traffic, resulting in a comprehensive 24 GB dump file containing 155.2 million packets for analysis. The labeling process enriches the data trace with information about the type, structure of the malicious traffic, and dependencies between single isolated malicious activities. The processing of the dumped data and logs, collected over a period of 6 days, resulted in 14.2M flows and 7.6M alerts. All the labeled traffic is related to the monitored services. Because the data collection process wasn't manipulated by injecting attacks or advertising on hacker chats, the dataset reflects real-world network conditions, ensuring the authenticity of the attacks captured. One of the main proposed enhancements to this strategy would be monitoring servers with varied user access, ensuring datasets contain both normal and malicious traffic in balance [69].

ISCX2012: The University of New Brunswick has supplied essential datasets for cybersecurity research, including ISCX IDS and ISCX Botnet. These datasets are vital for validating simulation models. The ISCX IDS dataset, in particular, provides realistic traffic for validation purposes. However, the ISCX-IDS2012 dataset has limitations. It lacks real-world attack distribution and does not adequately represent HTTPS flows. Additionally, payload recording in the UNB ISCX 2012 dataset is inconsistent [70].

ADFA: This dataset comprises two separate sets known as ADFA-LD and ADFA-WD, introduced in [71]. ADFA-LD is gathered from a modern Linux local server environment, specifically configured with Ubuntu 11.04 operating system, Linux kernel 2.6.38, and various enabled functions such as file sharing, database, remote access, and web server. It includes default installations of FTP, SSH, MySQL 14.14, Apache 2.2.17, PHP 5.3.5, and TikiWiki 8.1. The attacks against this host are carefully selected to simulate a realistic environment, including brute force password guessing attempts, unauthorized user creation, and compromise through payloads and webshells. ADFA-WD is a collection of DLL access requests and system calls for hacking attacks on a Windows XP SP2 host. It was gathered using Procmon, with the default firewall enabled and Norton AV 2013 installed to filter sophisticated attacks. The host environment includes file sharing, network printer configuration, and various applications like web server, database server, FTP server, streaming media server, and PDF reader. Exploitation of 12 known vulnerabilities was done using Metasploit framework and custom methods [72].

CICIDS2017: The dataset contains both benign and common attack traffic, reflecting real-world scenarios. It includes labeled network flows analyzed with CICFlowMeter and background traffic generated using the B-Profile system across various protocols. The data spans five days, starting from Monday, July 3, 2017, with benign traffic, and featuring attacks like Brute Force FTP, Brute Force SSH, DoS, and others on subsequent days [55]. Although this dataset is the most crucial for the research presented, it still has several notable shortcomings as mentioned in [73]:

- **Scattered Data:** The dataset is initially split across eight separate files, which complicates processing as merging these into a single file is required but time-consuming.
- **Large Volume:** Once combined, the dataset's size is significantly large, posing challenges in terms of data management and resource consumption during loading and processing.
- **Missing Values:** There are significant instances of missing data within the dataset, including 288,602 instances with missing class labels and 203 instances with incomplete information, necessitating data cleaning to achieve a finalized dataset with 2,830,540 unique instances.
- **Class Imbalance:** The dataset suffers from severe class imbalance, with the majority class (Benign) making up 83.34% of the data, and the minority class (Heartbleed) appearing in only 0.00039% of instances. This imbalance can lead

to biases in model training, resulting in diminished accuracy and increased false alarms when using the dataset to train detectors.

These issues highlight the limitations within the CICIDS2017 dataset that can affect its effectiveness and accuracy for training classifiers or detectors in cybersecurity research.

Table 2. Comparison between major public Datasets

Dataset	Entity	Year	Features	Threats	Network	Traffic	Labeled	Size (GB)
DARPA	Lincoln Laboratory	1998	41	Dos, R2L, U2R, Probe	Yes	No	Yes	4
KDD99	University of California	1999	41	Dos, R2L, U2R, Probe	Yes	No	Yes	0.7
DEFCON	The Shmoo Group	2002	CTF Logs	Port scan, Bad packets, Administrative privilege, Telnet protocol attacks.	No	No	No	-
CAIDA	Center of Applied Internet Data Analysis	2007	20	DDos	Yes	Yes	No	21
LBNL	Lawrence Berkeley National Laboratory	2004-2005	packet header traces	Malicious traces (no payload)	Yes	Yes	No	11
CDX	USMA	2009	Network & server logs	Dos, Scan, DNS	No	No	No	13
Kyoto	Kyoto University	2009	24	Honeypot traffic	Yes	No	Yes	0.55
Twente	Twente University	2009	Traffic flow	DoS, DDoS, Bruteforce, Infiltration	Yes	Yes	Yes	-
ISCX2012	University of New Brunswick	2012	Network traces	DoS, DDoS, Bruteforce, Infiltration	Yes	No	Yes	1
ADFA	University of New South Wales	2013	System call traces	FTP and SSH password BF, Java Meterpreter, Linux Meterpreter payload, C100 Webshell attacks.	Yes	Yes	Yes	20
CICIDS-2017	Canadian Institute of CyberSecurity	2017	80	Brute force, Portscan, Botnet, Dos, DDoS, Web, Infiltration	Yes	Yes	Yes	51.1

4.2. Generation Setup

From the analysis above, it is evident that most major public intrusion detection system datasets exhibit various shortcomings yet possess distinct advantages. To address these limitations and to harness all the benefits learned through years of trials, we decided to conduct a data generation experiment using our proposed architecture, which integrates multiple strategies to enhance the outcome. Below, we outline the methodology adopted for our experimental setup:

Data Conversion Tools: We utilized the CICFlowMeter tool, previously discussed and implemented for transforming pcap files into flow records in the CICIDS2017

dataset. The high quality of the features extracted from these packet capture files justifies our choice of the CICFlowMeter tool.

Environment Setup: Our experimental environment was established following the setup used in the ADFA-LD dataset. We configured a local Linux server (Ubuntu 22.04), which served multifunctional roles, including a file-sharing server, web server, personal computing device, and remote access server. This server was subjected to various attacks that will be detailed subsequently.

Addressing Class Imbalance: To tackle the issue of imbalanced classes, we redistributed the larger attack categories into multiple sub-categories and selected a diverse array of attacks to ensure comprehensive coverage. Furthermore, we aimed to minimize data redundancy by limiting the collection of records from highly repetitive attacks, such as DoS attacks, which do not significantly contribute new information and could detract from the model's accuracy for other classes.

Data Consolidation and Size Management: We streamlined the dataset to a more manageable size and consolidated our data into a single file, specifically designed for machine-learning applications.

Detailed Documentation: A comprehensive description of the dataset generation and the specific attacks performed is provided. This documentation is crucial for the ongoing generation of new data suitable for a federated learning approach and the continuous fine-tuning of large language models.

This structured approach to data generation not only facilitates the improvement of intrusion detection systems but also aligns with advanced learning methodologies that require systematic and continuous data input which should be the focus of our third experiment detailed in section 5.3.

4.3. Attack Scenarios

This section provides an analysis of the various attack methodologies used and their execution.

Brute Force - FTP:

In cybersecurity, brute force attacks leverage trial and error to decipher passwords, login credentials, and encryption keys. This straightforward approach effectively gains unauthorized access to individual and organizational systems. The attacker tests numerous combinations of usernames and passwords, typically automated by software, to identify the correct credentials.

The term "brute force" reflects the method's reliance on intensive, forceful attempts to breach accounts. Although an old technique, brute force remains effective and widely used among hackers. In the scenario we analyze, the attack targets a remote File Transfer Protocol (FTP) server after compromising a username. The attack tests a list of the most common 10,000 passwords, which reflect global user habits and demonstrate that despite its simplicity, brute force can be effective when paired with

statistical data on password usage. For this attack, we use Patator, a multi-purpose brute-forcing tool noted for its modular design and flexible application, supporting a variety of modules. This tool is adept at conducting brute force attacks efficiently.

Brute Force - SSH:

we apply the same scenario previously described, using the same tool and password list, but our objective is to gain access to a remote server via the Secure Shell Protocol (SSH).

SQL Injection:

To simulate an insecure web server scenario, we utilize WebGoat, a purposely insecure web application developed by the Open Web Application Security Project (OWASP). OWASP is a community-driven nonprofit organization dedicated to enhancing software security. WebGoat serves as an educational tool for learning about web application vulnerabilities, including SQL injection attacks, by providing a controlled environment for practical exposure and experimentation.

For this scenario, we employ sqlmap, an open-source penetration testing tool that automates the process of detecting and exploiting SQL injection flaws in web applications. SQL injection represents a critical security flaw that occurs when an attacker can inject malicious SQL statements into a database query via input fields of a web application. Assuming no prior external knowledge of the targeted system, the attack process begins with the use of Burp Suite, a cybersecurity testing toolkit developed by PortSwigger. This tool is primarily used for web application security testing and functions by intercepting and analyzing the data exchanged between a user and the web server. The information gathered during this phase aids in the identification of potential SQL injection points. Once a probable SQL injection vulnerability is identified, we transition to the enumeration phase. During this stage, we determine the type of Database Management System (DBMS), the structure of the database, and other critical information pertaining to the database server. Following the collection of this essential data, the SQL injection vulnerability is exploited to perform various operations, such as extracting sensitive information, including the identity of the current user connected to the application.

Cross-Site Scripting (XSS):

For this attack, we employed various targets and tools. Our targets included the a hosted Damn Vulnerable Web Application (DVWA). Additionally, we developed a simple, vulnerable one-page web application to explore multiple exploitation methods, such as direct injection of URL parameter values into HTML content without proper sanitization. For automated attacks, we utilized XSSStrike [74], featuring four hand-written parsers, an intelligent payload generator, a robust fuzzing engine, and a fast crawler. Unlike conventional tools that inject payloads and check their functionality, XSSStrike analyzes responses with multiple parsers and crafts contextually appropriate payloads, ensuring their effectiveness. XSSStrike also offers crawling, fuzzing, parameter discovery, WAF detection capabilities, and scans for DOM XSS vulnerabilities. Furthermore, we employed the Damn Small XSS Scanner (DSXS) to detect XSS vulnerabilities in our targets.

Portscan:

We use the Nmap tool to scan our target host, uncovering open ports and gaining insights into the network's topology and the services it hosts. This network reconnaissance phase is integral to cybersecurity operations, aiding in information gathering. To diversify our attack approach, we employ various connection methods such as TCP Connect Scan and SYN Scan. Furthermore, we leverage Nmap's powerful scripting engine to automate our attacks and extend the tool's functionality. For instance, we utilize SMB (Server Message Block) vulnerability scripts to assess the security status of the service on the target system.

Dos:

A Denial of Service (DoS) attack, while an age-old tactic, remains highly effective. It involves maliciously disrupting the normal operations of a server, service, or network by flooding it with an overwhelming volume of illegitimate traffic, requests, or connections. The primary objective is to deplete the target's resources such as bandwidth, processing power, or memory and rendering it inaccessible to legitimate users. This can lead to downtime, service disruptions, or the complete unavailability of the targeted system or network.

To understand and mitigate the impact of such attacks, we simulate and test various methods:

1. **Hulk:** Employing HULK (HTTP Unbearable Load King), this method generates a unique pattern for each request, evading detection. By ensuring dynamic responses and avoiding caching, it escalates the load on the target. Additional features include referer request obfuscation and masking the actual agent/actor.
2. **Low Orbit Ion Cannon:** LOIC is an open-source application designed for network stress testing and conducting denial-of-service (DoS) attacks. Developed in C#, LOIC functions by sending a large volume of TCP, UDP, or HTTP requests to the designated server. Specifically, we utilize LOIC to execute a TCP Flood attack, which involves establishing numerous half-open connections that strain resources on the target server while awaiting completion of the handshake process. Additionally, we use LOIC to execute a UDP Flood attack, whereby it transmits a continuous stream of UDP packets to random destination ports on the target.
3. **Slowloris:** Slowloris executes a denial-of-service attack by establishing and maintaining numerous simultaneous HTTP connections between the attacker and the target server. This tactic is illustrated in Figure 4.

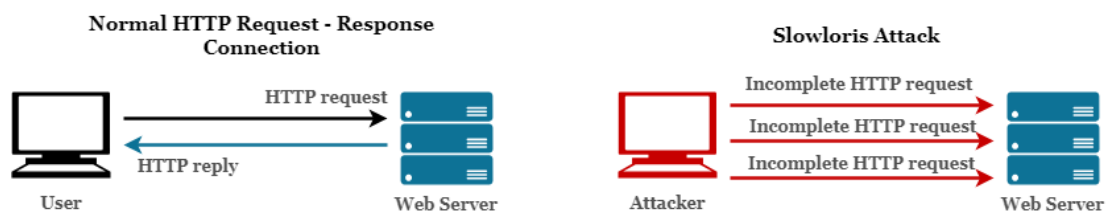


Figure 4. Slowloris Attack

4. **SlowHTTPTest:** This highly adaptable tool replicates several application layer Denial of Service attacks. In addition to Slowloris, it can execute other assaults like Slow HTTP POST and Slow Read attacks (exploiting the TCP persist timer). Moreover, it can simulate the Apache Range Header attack, inducing significant memory and CPU usage on the server.

Infiltration-Backdoor:

This multi-step attack is orchestrated using the Metasploit Framework Console. Initially, we generate two distinct versions: one tailored for Ubuntu systems and another in Python for general usage using our host's IP address and the port for communication to establish successful payload communication. Following this setup, we initiate a simple HTTP server on our attack machine. We then proceed to download this payload onto our victim machine, simulating the behavior of an unsuspecting user who may download a file without thorough scrutiny. Once the payload or code is executed on the victim machine, active communication is established between the attacker and the victim system. This enables us to execute commands on the victim machine with success.

Botnet:

Ares is a Python Remote Access Tool that comprises two primary components: a Command and Control (C2) server, accessible via a web interface for administering agents, and an agent program deployed on compromised hosts to establish communication with the C2 server. Utilizing Ares, we run a server on our attack machine to facilitate communication with the agent deployed on the victim machine.

DDos:

A distributed denial-of-service (DDoS) attack aims to disrupt the normal traffic of a targeted server, service, or network by overwhelming it with a flood of Internet traffic. This attack method achieves its effectiveness by leveraging multiple compromised computer systems as sources of attack traffic. These networks comprise infected computers and other devices, such as IoT devices, which have been compromised with malware, enabling remote control by the attacker. These individual devices are commonly referred to as bots.

To execute this attack, the creation of a botnet is essential. As discussed previously, we will utilize the Ares tool, as depicted in Figure 5, to construct our botnet. For this specific attack scenario, we will augment our closed network by adding three additional machines. Then, we will simultaneously execute the slowhttpstest attack from all agents on our target machine.

Name	Last Online	User	Host	IP	OS	Geolocation	Change name	Sel.
tarek_268263092083323	ONLINE	tarek	tarek-virtual-machine	192.168.43.131	Linux 6.5.0-26-generic	Local	Change name	☐
kali_52239283570	ONLINE	kali	kali	192.168.43.129	Linux 6.6.9-amd64	Local	Change name	☐
kali_52240860530	ONLINE	kali	kali	192.168.43.132	Linux 5.18.0-kali5-amd64	Local	Change name	☐

Figure 5. Botnet agent list

Path-Traversal:

DirBuster is used to conduct brute force attacks on directories and filenames within web applications. This tool operates by utilizing pre-existing files containing potential file and directory names. The lists were generated from scratch, by crawling the Internet and collecting the directory and files that are actually used by developers with a total of 9 different lists, it can also work with pure brute force. We target our two vulnerable applications, DVWA and Webgoat.

SMB-Analysis:

We establish a Samba file server on our victim machine for cross-platform file sharing over networks. Then, we utilize SMBMap to enumerate Samba shares across domains, listing drives, permissions, contents, enabling upload/download, auto-downloading files by pattern matching, and executing remote commands.

4.4. Analysis

Our dataset consists of a total of 77,666 instances categorized into 15 different labels. Each instance includes 79 distinct features, along with a final label, totaling 80 columns. The traffic features are extracted using CICFlowMeter and have undergone thorough filtering processes before being consolidated into the final dataset. Multiple scripts were employed to filter out only the attack records generated from the attack machine directed towards the target machine during the attack, with manual supervision ensuring accuracy. This resulted in a reduction in the number of records but improved efficiency in labelling. The class prevalence ratio of the labels in our dataset is presented in Table 3, while the distribution of labels is depicted in Figure 6. The CICFlowMeter is a flow-based feature extractor capable of extracting 80 features from a pcap file. In this dataset, the flow labels have been anonymized by excluding the SourceIP and DestinationIP, and then grouped into a single CSV file to facilitate their use in machine learning applications.

Table 3. Class prevalence ratio of our Dataset

Label	Number of instances	% of prevalence w.r.t. total instances	% of prevalence w.r.t. majority class
Normal	55149	71.007906	100.000000
DDos	4859	6.256277	8.810677
SQL-Injection	3845	4.950686	6.972021
Dos-Hulk	3073	3.956686	5.572177
Dos-Slowloris	2336	3.007751	4.235798
Dos-Slowhttptest	2174	2.799166	3.942048
FTP-BruteForce	1854	2.387145	3.361802
Ares-Botnet	1685	2.169547	3.055359
SMB-Analysis	751	0.966961	1.361765
Path-Traversal	668	0.860093	1.211264
SSH-BruteForce	520	0.669534	0.942900
Nmap-Portscan	256	0.329617	0.464197
Cross-Site-Scripting	246	0.316741	0.446064
Dos-LOIC	159	0.204723	0.288310
Infiltration-Backdoor	91	0.117168	0.165008

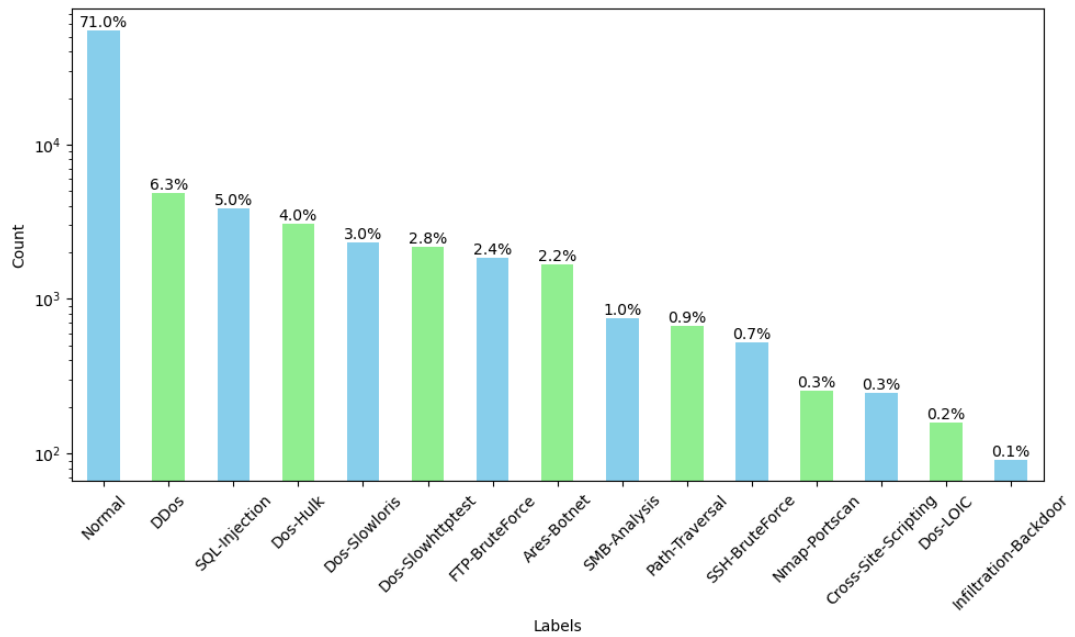


Figure 6. Label Distribution

For the next step of our analysis, we have utilized three fundamental metrics in information retrieval evaluation:

1. Precision (Pr):

Precision quantifies the accuracy of the positive predictions made by a classifier or system. It is calculated as the ratio of true positives to the sum of true positives

and false positives. In other words, precision measures the proportion of relevant instances among the retrieved instances. Mathematically, it is represented as:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

A high precision indicates that the classifier retrieved relevant instances with a low rate of false positives.

2. Recall (Rc):

Recall, also known as sensitivity or true positive rate, gauges the ability of a classifier to correctly identify all relevant instances. It is calculated as the ratio of true positives to the sum of true positives and false negatives. In essence, recall measures the proportion of relevant instances that were retrieved by the classifier. The formula for recall is given by:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

A high recall indicates that the classifier successfully identified most of the relevant instances, minimizing the rate of false negatives.

3. F-Measure (F1):

The F-Measure, often referred to as F1 score, harmonizes precision and recall into a single metric. It is the harmonic mean of precision and recall, providing a balanced assessment of a classifier's performance. The formula for F1 score is:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1 score ranges from 0 to 1, where a value of 1 indicates perfect precision and recall, while 0 indicates the worst performance. F1 score is particularly useful when there is an uneven class distribution or when both precision and recall are important.

These evaluation metrics offer insights into a classifier's effectiveness in information retrieval tasks, enabling a thorough assessment of our dataset's utility in machine learning and anomaly classification applications. We've provided these metrics using both weighted average and macro average.

- **Weighted Average:** Accounts for class imbalance within the dataset, weighting each class's contribution based on its prevalence.
- **Macro Average:** Treats each class equally, computing metrics independently for each class and then taking the unweighted mean.

In summary, while weighted average considers class distribution, macro average treats each class equally, irrespective of its size, when computing the overall metric. The complete results are presented in Table 4. Based on the analysis of the provided data, it is observed that a majority of the models demonstrate acceptable performance when applied to our dataset. Considering criteria such as execution time, computational complexity, and compatibility with the explainability frameworks utilized in our experiments, we have made the decision to employ the Random Forest classifier. This selection is driven by its ability to deliver high accuracy, efficient inference, and easily interpretable outcomes.

Table 4. Performance Metrics of Machine Learning Models

Algorithm	Pr (macro)	Pr (weighted)	Rc (macro)	Rc (weighted)	F1 (macro)	F1 (weighted)	Accuracy
RF	0.99	0.98	0.98	0.98	0.99	0.98	0.979
KNN	0.98	0.97	0.97	0.97	0.97	0.97	0.970
ID3	0.98	0.98	0.98	0.98	0.98	0.98	0.978
GBM	0.98	0.98	0.98	0.98	0.98	0.98	0.979
MLP	0.97	0.96	0.97	0.95	0.97	0.96	0.955
GNB	0.88	0.89	0.90	0.86	0.87	0.86	0.856
QDA	0.95	0.92	0.91	0.90	0.92	0.89	0.899
SVM	0.91	0.91	0.90	0.90	0.90	0.90	0.901

4.5. Shortcomings

While we maintain confidence in the quality of our work, we acknowledge several areas for potential improvement in future endeavors, including:

- **Limited diversity in network targets:** To ensure dataset accuracy and specificity for our particular use case, we intentionally restricted our target machines to those running Ubuntu systems. However, a potential direction for improvement involves expanding our work to include a broader range of machine types and operating systems.
- **Labeling only the CSV flow data:** Our focus primarily revolves around providing a dataset tailored for modern Network Intrusion Detection Systems (NIDS) applications. Consequently, we have solely labeled the final CSV-processed flows, omitting the labeling of the original PCAP files. However, for a more comprehensive analysis, future iterations may benefit from including labeling for both the original PCAP files and the processed CSV flow data.

5. EXPERIMENTS

5.1. Experiment 1: Integration of Explainable AI with LLMs

In this section, we narrow our focus to provide more detailed insights into the architecture of HuntGPT, engaging in a concise discussion on each primary component. Figure 7 depicts a comprehensive illustration of the diverse components comprising our Intrusion Detection System Dashboard.

Our IDS Dashboard enhances trust in the AI system by revealing model insights and contributing features. It lets users inspect original data packets for manual anomaly inspection. AI-generated explanations further clarify predictions and suggest appropriate course of action. The dashboard also facilitates interactive discussions with the AI assistant, offering custom insights into the model's operations. This component integrates several sub-modules to facilitate its work in presenting explainability and fairness to the end user.

1. **OpenAI Connector:** The OpenAI Connector is mainly used for authentication with the OpenAI API and also initiates the prerecorded message history. It also keeps track of the user conversations.
2. **Anomaly Packet Data Fetching:** It looks through all the documents in Elasticsearch index and extracts all the important information we need from that document.
3. **OpenAI API Unit:** Integrates the detected packet flow data with the curated fine-tuning prompts and feeds them to the OpenAI API.
4. **AI Assistant Analysis:** The AI Assistant receives all the data preserved in the document, accompanied by the refined prompts from the unit, and it generates a comprehensive analysis for the human agent. This analysis not only reveals the details but also facilitates interactive communication directly with the human agent, enabling a seamless exchange of information and insights.

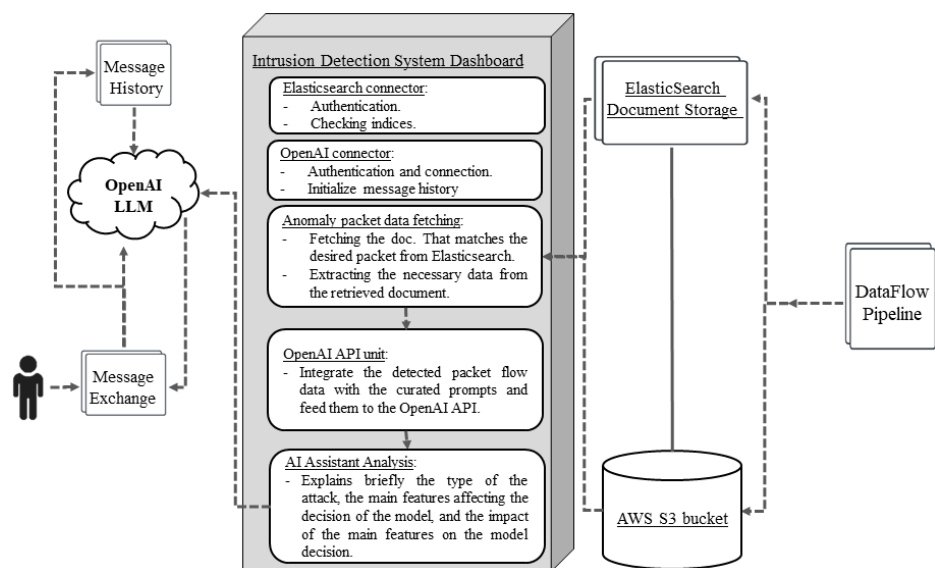


Figure 7. HuntGPT components Diagram

5.1.1. Anomaly Detection Model

The model driving our network anomaly detection is a Random Forest classifier trained and tested on our generated Dataset, achieving an accuracy of 0.98.

5.1.2. Elasticsearch Data Schemas

For telemetry storage, we principally utilize two Elasticsearch indices: detected-packets, original-packets. The "detected-packets" index serves as a structured repository for crucial information related to detected network anomalies. This schema is designed to store the attributes extracted using the two explainability frameworks SHAP and LIME, each contributing to the comprehensive understanding of identified anomalies.

The schema for the "detected-packets" index is designed to align with the objective of offering insights into the workings of the anomaly detection model, ensuring transparency and a deeper understanding of its mechanisms. It offers a structured means of storing textual predictions, influential factors, and other data, enhancing the dashboard's capabilities to explain anomalies effectively. Table 5 provides an overview of the "Detected Packets" index, detailing each label and its corresponding description to ensure clarity and a comprehensive understanding of the data structure. The table emphasizes key attributes pivotal for anomaly detection and subsequent analysis.

Table 5. Detected Packets Index

Label	Description
_id	Represents a unique identifier for each detected packet entry.
Prediction	Stores the prediction outcome associated with the detected packet, whether normal or malicious and also the specific genre of the attack.
Factors	Captures the contributing factors that influenced the prediction outcome and the key attributes that led to the anomaly's detection. We store textual descriptions of these factors, to be later translated into visual plots and also fed into the chatbot to provide contextual answers.
Exp-img	Stores references to the plots used in presenting the prediction process, enhancing the comprehensibility of the anomaly detection process. Exp-img represents the local explanation plot provided by LIME tabular explainer.
Shap-img	Shap-img represent the top features contributing to the predicted anomaly class using SHAP framework.
Original-data	Denotes the number of the detected packets which can be used as a foreign key to access the complete information regarding the detected packets.

The "original-packets" index requires a more complex and dynamic mapping. An in-depth review of each field in the document is presented in the following section 4 where

we get into the details of the dataset creation process. The schema for the original packets is crucial, aiding anomaly detection through manual reviews and enhancing trust between the human agent and the IDS.

5.1.3. Evaluation

Collectively, all the elements of the IDS Dashboard we previously discussed assist the user in making well-informed decisions regarding network anomalies by incorporating explanatory visualizations, manual inspection, AI-generated explanations, and interactive conversations. The system is intended to be versatile and can aid the user in multiple scenarios.

Prototype Functionality

The dashboard shown in Figure 8 features a neatly organized layout that seamlessly integrates the multiple components of the HuntGPT. In the upper section, the dashboard provides a visual representation of the ML model inner workings. Moving down, the middle area presents the original data for manual inspection when needed. Lastly, the lower part is the AI assistant that provides context and explainability to the end user and also enables on going conversations. The system user can download a complete report of the incident, including all the graphs and data regarding the incident using the generate report function. This setup ensures user-friendliness while uniting various functions.

Response Quality Analysis

In this section, we examine how well our prototype functions in explaining detected anomalies and assisting users via its chatbot feature. Assessing the responses of the chatbot, a common practice with conversational agents, posed distinctive challenges in quantifying the requisite metrics for performance appraisal. Our analysis can be segmented into two key components: i) Technical Knowledge in Cybersecurity and ii) Response Evaluation.

We first examine whether ChatGPT (i.e., GPT-3.5 Turbo) possesses the requisite technical knowledge in the field of cybersecurity to effectively assist the user. A critical aspect of this analysis involves comparing ChatGPT's knowledge to that of a certified IT professional. Following this knowledge assessment, we evaluate the responses provided by ChatGPT in terms of their quality and appropriateness. Special attention is paid to the level of difficulty in the answers since our goal is to provide knowledge to users with minimal cybersecurity experience.

Technical Knowledge in Cybersecurity

In this assessment, our primary focus is to measure the accuracy and precision of answers generated by GPT-3.5-turbo, the model currently powering the HuntGPT system. To evaluate the model's performance, we conducted tests using a set of



Figure 8. Sections of Detection and Explainability in the Dashboard.

standardized certification exams in the field of cybersecurity. The detailed list of the standardized exams used for our assessment is presented below:

1. CISM Certified Information Security Manager Practice Exams: The updated self-study guide Written by Peter H. Gregory featuring hundreds of practice exam questions that match those on the live test [17].
2. ISACA official CISM practice Quiz. A free practice quiz includes questions from ISACA’s test prep solutions that are the same difficulty level as ISACA’s official CISM exam [75].
3. ISACA official cybersecurity fundamentals practice quiz: A practice quiz including questions from ISACA’s test prep solutions that are the same level of difficulty of ISACA’s official Cybersecurity Fundamentals exam [75].

Based on the results presented in Table 6, GPT-3.5-turbo demonstrates substantial proficiency in cybersecurity knowledge, with success rates ranging from 72% to 82.5% across varied and reputable exams. However, there is room for improvement, particularly in the ISACA official cybersecurity fundamentals practice quiz where the model achieved a lower success rate of 72%. The challenging CISM certification demands profound knowledge and experience in risk management, illustrated by the fact that only 50-60% of first-time examinees attain success [76]. Nevertheless, preliminary results from our evaluation are highly promising and indicate that the model possesses the capacity to provide well-informed security decisions.

Table 6. Comparison of CISM and Cybersecurity Exam Preparations: GPT-3.5 Success Rates

Exam	No. of Questions	GPT-3.5 turbo Success Rate
CISM Certified Information Security Manager Practice Exams [17]	40	82.5%
ISACA official CISM practice Quiz [75]	10	80%
ISACA official cybersecurity fundamentals practice quiz [75]	25	72%

Response Evaluation

We evaluated the responses provided by GPT-3.5-turbo in terms of their quality and appropriateness. Table 7 presents the comprehensive findings of our study, where we evaluated 20 different generated explanations and 20 chat logs. Each chat log included approximately six questions regarding the detected anomaly. An example of a chat log is illustrated in Table 8.

We evaluated the responses using six unique readability formulas from the Python *py-readability-metrics* library [77], each applicable to different fields due to their distinct characteristics. The Flesch-Kincaid Grade Level, adopted by the U.S. Army, is used for assessing the complexity of technical manuals. The Flesch Reading Ease serves as a standard test for the U.S. Department of Defense, employed to determine the readability of its documents and forms. The Dale Chall readability formula is widely adopted in schools and educational institutes to assess the difficulty

level of texts. The Automated Readability Index was initially designed for real-time monitoring of readability on electric typewriters. The Coleman Liau Index has been specifically used to evaluate the readability of medical documents. Finally, the Linsear Write was purportedly developed for the United States Air Force to assess the readability of their technical manuals.

The readability results in Table 7 for both AI explainability and Chatbot logs were evaluated by most formulas as being at the graduate level or equivalent, revealing the presence of complex linguistic structures in the generated texts. While chat conversations can display different readability levels depending on question types and user-required detail, the generated text is usually readable, typically necessitating only a basic, non-specialized college education level for comprehension.

Table 7. evaluating generated text readability

Readability Formula	Generated Anomaly explanation		Chatbot Answers	
	Score	Grade level	Score	Grade level
Flesch-Kincaid Grade Level	15.7	16	14.9	15
Flesch Reading Ease	22.7	graduate	23.9	graduate
Dale Chall	12.6	graduate	12.11	graduate
Automated Readability Index	16.3	graduate	15.9	graduate
Coleman Liau Index	15.3	15	16.3	16
Linsear Write	17.4	17	15.8	16

Table 8. Examples of submitted queries to HuntGPT and the related actionable responses

Query	Chatbot Response
How can I prevent such attack	To prevent DoS attacks, you can implement measures like traffic monitoring, firewalls, load balancers, and rate limiting to detect and mitigate abnormal traffic patterns. Ensuring network redundancy and having a robust incident response plan can also help minimize the impact.
What is a firewall and how do I implement it with some good firewall examples	A firewall is a network security device that filters incoming and outgoing network traffic based on predetermined security rules. Examples of good firewalls include: Cisco ASA (Adaptive Security Appliance), Palo Alto Networks Next-Generation Firewalls, Fortinet FortiGate Firewalls, Check Point Firewall. To implement a firewall, configure rule sets to allow or block specific types of network traffic and define security policies to secure your network from unauthorized access.
Are there any free ones?	Yes, there are free firewall solutions available. Some popular free firewall options include: ZoneAlarm: ZoneAlarm offers a free version of their firewall software for personal use, providing basic firewall protection along with additional features like identity protection and anti-phishing. Windows Firewall: If you are using a Windows operating system, it comes with a built-in firewall called Windows Firewall. It provides basic inbound and outbound traffic filtering capabilities. These free firewall solutions can offer varying levels of protection and features.
How can I install zonealarm	To install ZoneAlarm, follow these steps: Visit the official ZoneAlarm website (www.zonealarm.com) using your web browser. Go to the "Products" or "Downloads" section of the website. Look for the free version of ZoneAlarm Firewall. [proceeds to give detailed instructions on downloading ZoneAlarm]
What would be a good configuration for a small home network	To configure ZoneAlarm Firewall for a small home network: Install and update ZoneAlarm. Use medium or high-security mode. Enable automatic program control. Customize program permissions. Enable Stealth Mode for added security. Set up Network Zones. Monitor network traffic. Keep ZoneAlarm updated.

5.2. Experiment 2: LLMs As Classifiers

We propose a second experiment that involves fine-tuning multiple pretrained language models using our newly created dataset. The goal is to evaluate the effectiveness of these models in solving the multi-label dataset classification problem. This section outlines the methodology, detailing the dataset preparation, the models used, and the process for evaluating results.

Our work investigates the effectiveness of language models in applications beyond human-readable text, specifically in processing network flow records—a critical aspect of cyber operations. It addresses the challenge of integrating recent research to run natively within network infrastructure. Experimental findings demonstrate the effectiveness of LLMs in classifying network data, suggesting they are promising candidates for native AI solutions. Unlike traditional machine learning algorithms, LLMs require minimal preprocessing, enhancing intuitiveness and efficiency.

5.2.1. Data Balancing

After observing an imbalance in our data 4.4, we take steps to address it. The proportion of the minority class in the entire dataset is found to be 0.12%, indicating an extreme imbalance but also a resemblance to the nature of the problem in hand because the majority of network traffic is mostly normal and non malicious. Following this, we first start by decreasing normal traffic by 30% and then proceed to divide our data into training, validation, and test sets:

- **Training set size:** 31,248 records
- **Validation set size:** 6,836 records
- **Test set size:** 977 records

Next, we processed the training set without altering the evaluation or testing sets, ensuring they accurately represent normal network activity. After multiple experiments with oversampling, undersampling, and a combination of both, we arrived at our best-performing dataset using a downsampling approach. This method achieved a better balance in the dataset, eliminating the need for further balancing techniques. The final training set size is 13,441 records, with the distribution detailed in Table 9. No additional cleaning steps were necessary.

Table 9. Class prevalence after downsampling our Dataset

Label	Number of instances
Normal	2223
DDos	2198
SQL-Injection	1712
Dos-Hulk	1379
Dos-Slowloris	1040
Dos-Slowhttptest	970
FTP-BruteForce	826
Ares-Botnet	929
SMB-Analysis	597
Path-Traversal	530
SSH-BruteForce	418
Nmap-Portscan	211
Cross-Site-Scripting	202
Dos-LOIC	133
Infiltration-Backdoor	73

5.2.2. Models

We explore and compare a variety of models in our study such as classifiers and Sequence to Sequence models. We performed supervised training on different variations of BERT-based LLMs to classify data according to attack type. Below are the models we explored:

- **Bert Base Uncased:** BERT, short for Bidirectional Encoder Representations from Transformers, is a language representation model first introduced by devlin et al.[78] in 2018, designed to pretrain deep bidirectional representations from unlabeled text. It accomplishes this by simultaneously considering both left and right context in all layers. This pre-trained BERT model can then be fine-tuned with just one additional output layer, enabling the creation of state-of-the-art models for a wide range of natural language processing tasks.
- **Roberta base:** The model introduced in [79] presents a replication study of BERT and proposes an improved recipe for training BERT models, called RoBERTa, that can match or exceed the performance of all post-BERT methods. the modifications in this version are straightforward and include:
 1. Training the model for a longer duration, with larger batches, and over more data.
 2. Removing the next sentence prediction objective.
 3. Training on longer sequences.
 4. Dynamically changing the masking pattern applied to the training data.

This version is using the BERT-base architecture with 12-layers, 768-hidden, 12-heads, 125M parameters.

- **Roberta Large:** Another version of RoBERTa using the BERT-large architecture with 24-layers, 1024-hidden, 16-heads, 355M parameters. We can examine a comparison of the two models in figure.9.

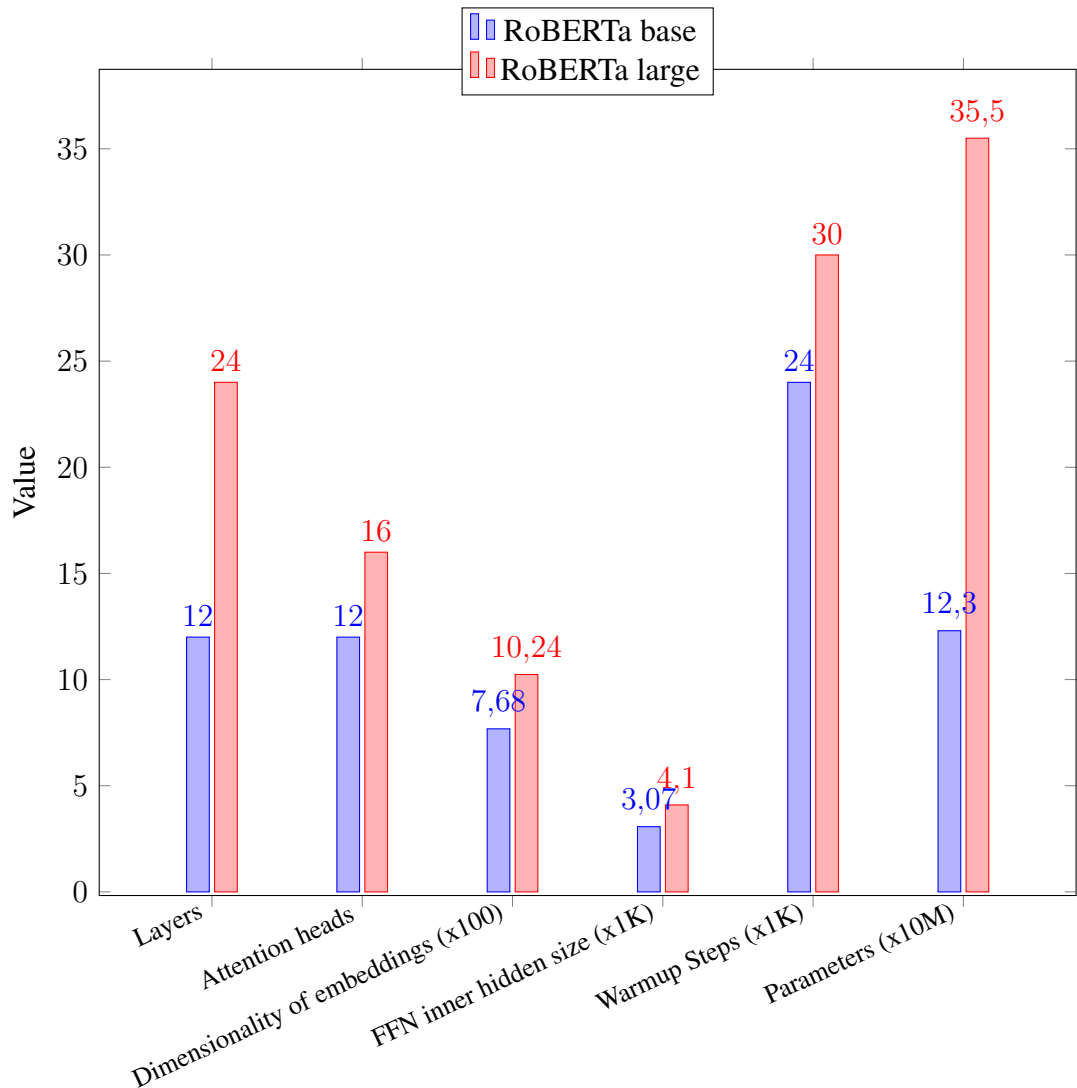


Figure 9. RoBERTa base Vs. RoBERTa large Hyperparams

- **Deberta base:** Decoding-enhanced BERT with disentangled attention, a model architecture that enhances BERT and RoBERTa using two key techniques. Firstly, employing disentangled attention, representing each word with two vectors for content and position, improving attention computation. Secondly, an enhanced mask decoder that incorporates absolute positions for predicting masked tokens during pre-training. also, a new virtual adversarial training method was used for fine-tuning to improve models' generalization[80].

For the Sequence-to-Sequence models, we investigate the T5 model, or Text-To-Text Transfer Transformer, as originally presented in [81]. This deep learning model is

designed to address a wide range of natural language processing tasks by framing them as text-to-text problems. T5 models are pretrained on an extensive dataset of text and code using this text-to-text framework, enabling them to perform the tasks for which they were initially trained. Additionally, these models can be fine-tuned to perform other specific tasks. The T5 architecture utilizes a standard encoder-decoder structure, where the encoder processes the input text and the decoder generates the output text. The T5 series comprises several models with varying sizes and capabilities as demonstrated in Table 10, differentiated by their parameter counts, which reflect the model’s complexity and potential performance capacity.

Table 10. T5 series models

Model	Parameters	No. layers	d_{model}	d_{ff}	d_{kv}	No. heads
Small	60M	6	512	2048	64	8
Base	220M	12	768	3072	64	12
Large	770M	24	1024	4096	64	16
3B	3B	24	1024	16384	128	32
11B	11B	24	1024	65536	128	128

No. layers: Number of layers in encoder and decoder.

d_{model} : Dimension of the embedding vectors.

d_{ff} : Dimension of the feedforward network within each encoder and decoder layer.

d_{kv} : Dimension of the key and value vectors used in the self-attention mechanism.

No. heads: Number of attention heads in each attention block.

5.2.3. Evaluation

To assess our models, we employ standard evaluation metrics during the model training process.

Classifier

Our preliminary tests demonstrate promising outcomes for our use case. In Table 11, we present the results of fine-tuning our models, while Figure 10 illustrates the loss values across the three training epochs. Initial observations suggest that all models yield satisfactory results, with the lowest performance from RoBERTa Large. Overall, these insights indicate that the BERT Base model generally outperforms the other models across all metrics, followed by the Roberta Base model. Further investigation reveals that RoBERTa Large requires additional training epochs and a larger dataset to achieve optimal performance, whereas the base model delivers better results.

Table 11. Summary of Classifier Models Metrics

Validation Metrics	Bert Base Uncased	Roberta Large	Roberta Base	Debert Base
loss	0.06196	0.10873	0.08128	0.07306
F1 macro	0.97876	0.95561	0.96899	0.96998
F1 micro	0.98406	0.97513	0.97937	0.98098
F1 weighted	0.98410	0.97535	0.97943	0.98099
Precision macro	0.97572	0.94776	0.96262	0.96506
Precision micro	0.98406	0.97513	0.97937	0.98098
Precision weighted	0.98536	0.97784	0.98148	0.98286
Recall macro	0.98342	0.96724	0.97859	0.97740
Recall micro	0.98406	0.97513	0.97937	0.98098
Recall weighted	0.98406	0.97513	0.97937	0.98098
Accuracy	0.98406	0.97513	0.97937	0.98098

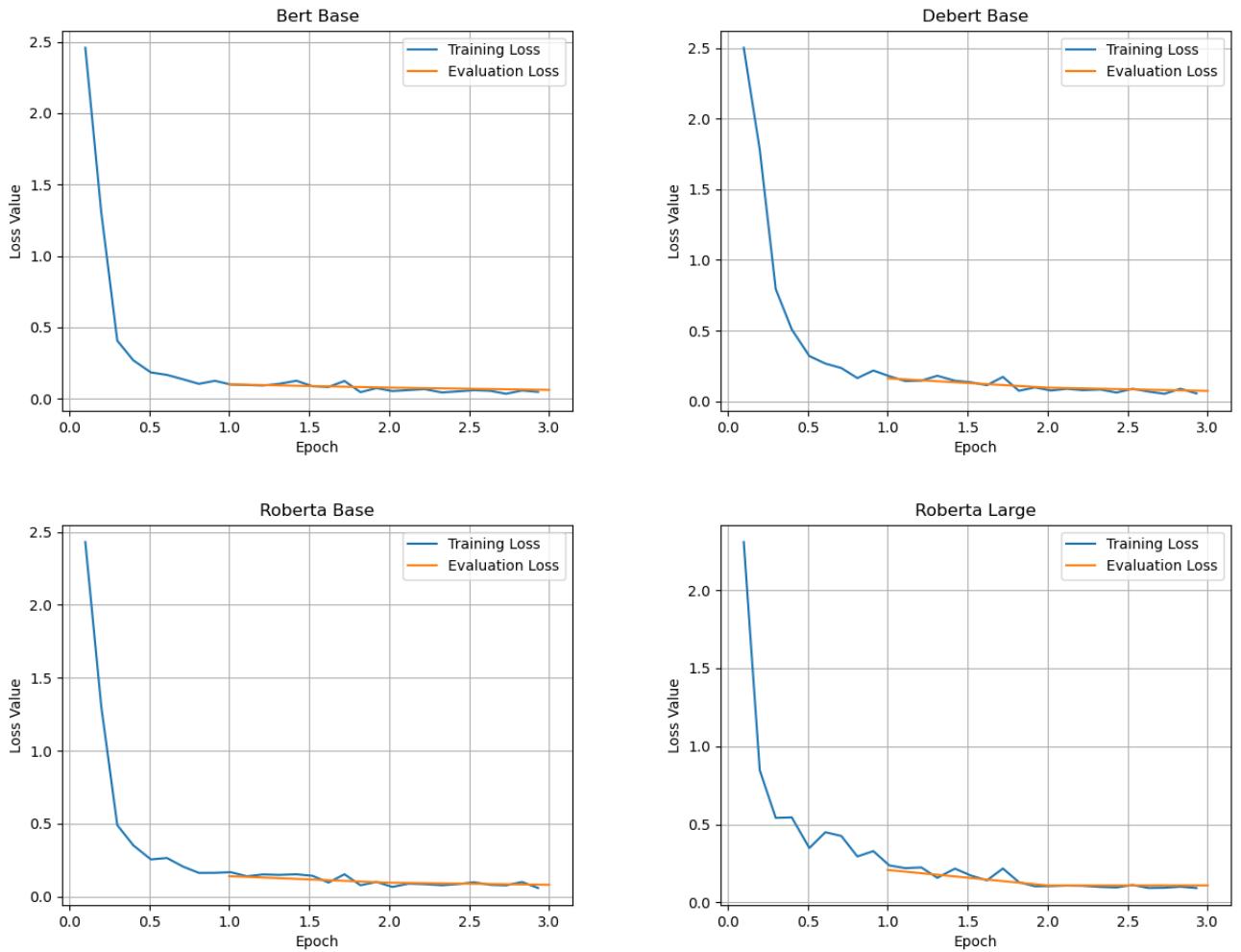


Figure 10. Model Loss During Training.

Sequence to Sequence

Our preliminary tests for these category of transformer models show promising results as well. Table 12 displays the fine-tuning results of our models, and Figure 11 shows the loss values over three training epochs. Initial observations suggest that all models perform well, with accuracy and performance improving as model sizes increase. Specifically, the T5 Large model achieves an impressive accuracy of about 98.5% on the test set.

Table 12. Summary of Seq-2-Seq Models Metrics

Validation Metrics	T5 Small	T5 Base	T5 Large
loss	0.034	0.021	0.014
ROUGE-1	97.338	98.384	99.020
ROUGE-L	97.367	98.384	99.013
Accuracy	96.315	97.850	98.465

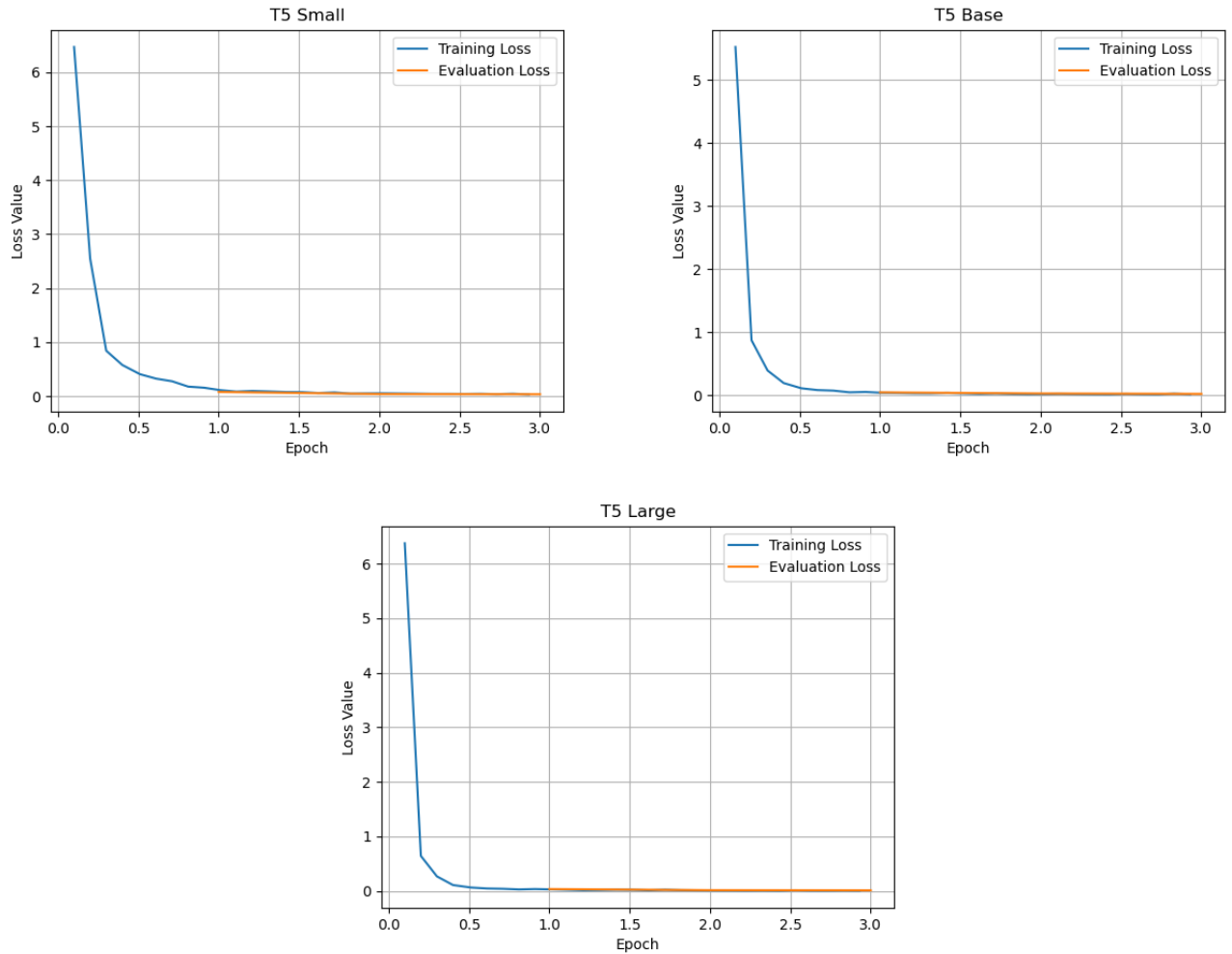


Figure 11. Model Loss During Training.

5.3. Experiment 3: Autonomous Data Regeneration for Augmentation

The primary motivation behind this experiment is to complete our end-to-end solution for network anomaly detection. We aim to cover three aspects of AI integration: Explainable, Actionable, and Interpretable Artificial Intelligence, following the Cybertrust framework [1] that guides our experiments development. This experiment addresses how actionable AI can tackle the challenges of increasing complexity and diverse network topologies and the generation of new data for retraining models.

Supervised anomaly-based Intrusion Detection Systems are crucial in maintaining network security. However, these systems suffer from high false positive rates when the network environment or services change. Supervised anomaly-based IDS are designed to identify deviations from established profiles of normal network behavior and predefined attack scenarios. These profiles are generated from historical data, capturing patterns of legitimate and illegitimate activity. However, when network environments or services change, these systems often mistakenly classify new, legitimate activities as malicious, resulting in a high rate of false positives. Some causes of high false positive rates include:

- **Static Profiles:** IDS profiles are built using historical data, making them inherently rigid and unable to adapt to new patterns of normal behavior[82].
- **Dynamic Network Environments:** Modern networks are highly dynamic, with changes in connectivity, configurations, and mobile nodes. This dynamic nature is not reflected in the static profiles and introduces new vulnerabilities that traditional, static security solutions struggle to address effectively. As network components are upgraded or new ones added, threat exposure increases. Continuous monitoring and adaptation to these evolving vulnerabilities are essential to maintaining effective security and reducing risks.
- **Service Changes:** Network services are essential components of modern networks, providing various functionalities such as data transfer, communication, and resource sharing. These services include web hosting, email, file sharing, database management, and cloud computing, among others. The introduction or modification of network services can create new types of legitimate traffic that differs in volume, frequency, and type from historical patterns and can be misclassified as anomalies.
- **New Attack Vectors:** As new types of attacks are developed, existing IDS profiles may not recognize them, leading to either undetected threats (false negatives) or misclassification of benign activities as threats (false positives).
- **Usage Variability:** Different organizations utilize networks in unique ways, influenced by their specific operational needs, permissions, and security policies which can result in legitimate activities being incorrectly flagged as anomalies.

Building on the previous analogy, we opted for a novel strategy that addresses two principal challenges. The first challenge is how to generate new data for retraining our intrusion detection system when deployed in diverse environments, while simultaneously identifying and accounting for the unique characteristics of each

network. The second challenge involved utilizing this newly generated data, which may not encompass all potential attack scenarios due to variations in network topology and services, in combination with our original dataset.

The objective is to synthesize a more comprehensive dataset that will combine the newly generated data within the network with our original dataset that contains a complete profile for all attacks performed manually to be utilized to retrain the model, thereby improving the overall quality of the data and improving the IDS performance.

5.3.1. *Sec-GPT*

Building upon the earlier discussion of LLM agents, we developed an interactive agent designed to enhance the penetration testing process for devices equipped with our CICFlowMeter to generate new attack profiles. This agent features several key components:

- **LLM Reasoning Capabilities:** The agent employs advanced natural language processing and reasoning abilities inherent in LLMs to comprehend and respond to various pentesting scenarios.
- **Domain-Specific Knowledge Base:** It includes a comprehensive knowledge base focused on cybersecurity and network analysis, ensuring contextually relevant insights and guidance.
- **Executable Toolset:** Equipped with a comprehensive toolkit, the agent is capable of executing a wide range of penetration testing activities. These activities span from vulnerability scanning to exploit execution, enabling the generation of diverse attack profiles.

Architecture

Our Sec-GPT follows the architecture illustrated in Figure 12. Interaction begins with the user selecting the desired model to interact with, which can include any OpenAI model, although our preference is GPT-4o due to its balance of code quality, cost-efficiency, and reasoning capabilities. The agent then guides the user through the required penetration testing procedures, following the provided prompts and its integrated knowledge base. our architecture is divided into two main sections:

- **Retrieval-Augmented Generation (RAG)**

The RAG section is designed to optimize and align the output of the LLM by referencing an external knowledge base beyond the training data sources before generating a response. Our RAG pipeline is implemented using Microsoft Azure's AI Search service[83], a cloud-based search service that enhances information retrieval in databases and documents. In our pipeline, it fetches relevant information by indexing, pulling, and ranking data from indexed sources to provide precise search results that inform subsequent AI-driven tasks.

Several documents detailing the process of scanning, attack strategies, and the various tools and capabilities installed with the system are stored in Blob

Storage. Blob Storage is a cloud-based service designed for storing large amounts of unstructured data, which we index and query.

The final RAG pipeline structure is as follows:

1. **Data Storage:** Context documents are stored in Blob Storage.
2. **Indexing:** AI Search indexes the data stored in Blob Storage.
3. **Retrieval:** When a query is made, the search service retrieves relevant information from the indexed data, including metadata and search scores, to ensure high-quality context.
4. **Augmentation:** The retrieved information is then fed into the LLM to generate the necessary code and steps.

The primary motivation for adopting a cloud-based RAG pipeline is its capability to continuously update the LLM's knowledge on new attacks, tools, and methodologies. This approach allows for ongoing enhancements without the need for system updates or redeployments. Moreover, this approach enables the potential deployment on edge nodes within modern 5G networks.

- **Local Deployment Framework**

The execution framework primarily consists of a local code execution unit that extracts and deploys code and commands based on LLM instructions. This unit includes a feedback loop that enables the LLM to identify and correct deployment errors, such as missing libraries, syntax errors, and other issues. Furthermore, the framework incorporates a bundle of pre-configured tools specifically designed to generate attacks on the target machine being tested.

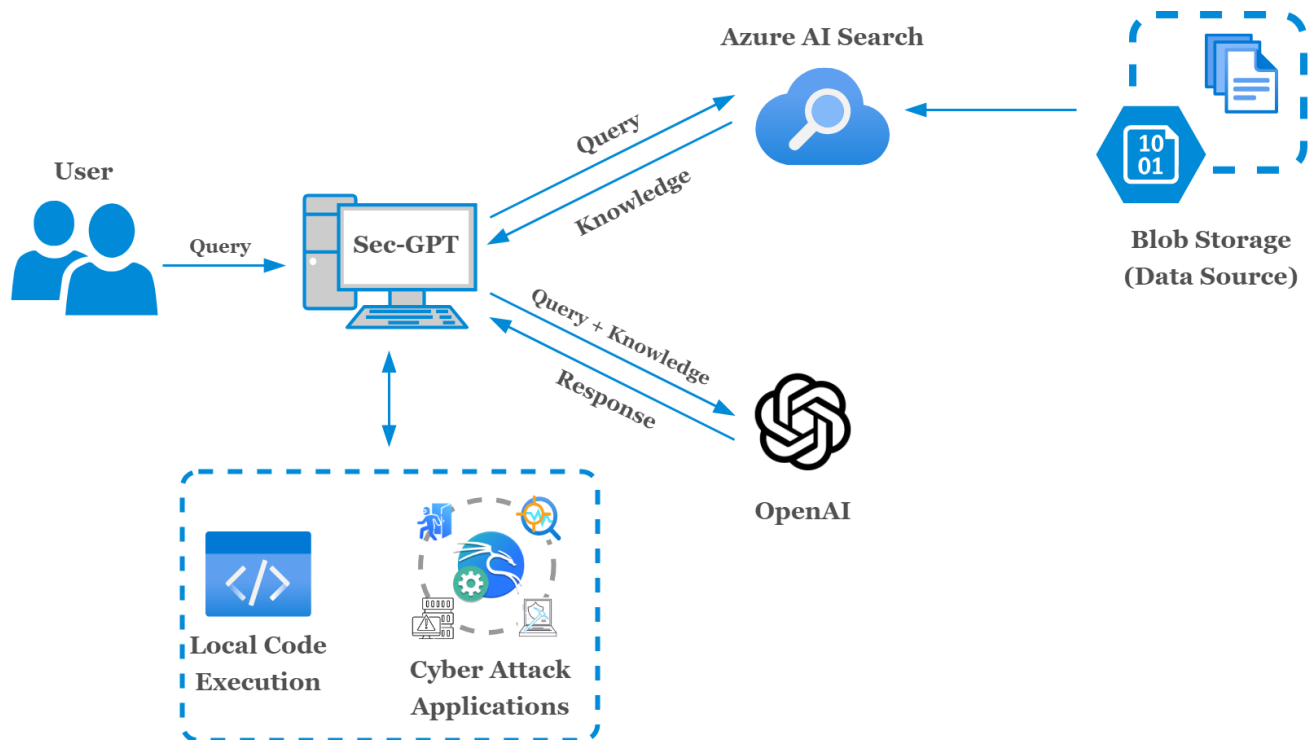


Figure 12. Sec-GPT Diagram

5.3.2. Synthetic Data Generation

Using synthetic data to augment the generated data provides a lot of benefits for our IDS models:

- **Class Balancing:** Synthetic data can enhance the existing dataset by addressing data scarcity and balancing class distribution, especially for rare attack types.
- **Data Privacy and Security:** Synthetic data preserves privacy by mimicking the statistical properties of real data without exposing sensitive information, making it safer for sharing and collaborative research.
- **Cost-Effective Data Generation:** It generates large, annotated datasets cost-effectively, bypassing the expensive, time-consuming process of collecting real-world data.
- **Environment-Specific Training:** Tailors synthetic data to specific network environments, improving detection in those contexts.
- **Handling Dynamic Environments:** Simulates diverse network conditions and evolving threats, ensuring IDS adapt to and effectively manage dynamic, real-world environments.
- **Continuous Learning and Adaptation:** Provides up-to-date synthetic data that aids continuous retraining and adaptation of IDS, maintaining their effectiveness against emerging threats.
- **Robustness to Noise and Variability:** Trains IDS on varied, noisy data to improve robustness, reducing false positives and negatives in variable real-world conditions.

In summary, synthetic data significantly enhances IDS to become more robust, adaptive, and effective across various dynamic and evolving requirements.

Generative Adversarial Networks

Generative Adversarial Networks (GANs), proposed by Ian Goodfellow et al. in 2014, presents a unique adversarial approach to generative model estimation. Utilizing two simultaneously trained models, a generative model (G) that captures the data distribution, and a discriminative model (D) to estimate the likelihood of a sample originating from training data versus G. The training strategy for G is maximize the probability of misclassification by D, hence embodying a two-player minimax game [84].

In the training process, the discriminator model is initially trained with a combination of both authentic and synthesized data, while maintaining the gradients of the generator model steady. This learning process is then used to backpropagate the gradients calculated by the discriminator. Following this, the objective of the Generator is to create data convincing enough for the Discriminator to classify it as authentic, with the gradients backpropagated from the Discriminator. The standard

model structure and the training process are generally illustrated in Figure. 13. GANs have demonstrated remarkable advantages in synthetic data generation across varying fields. They are extensively utilized in Image Synthesis through generating high-quality images. They can also effectively enhance datasets, particularly when the original data is insufficient.

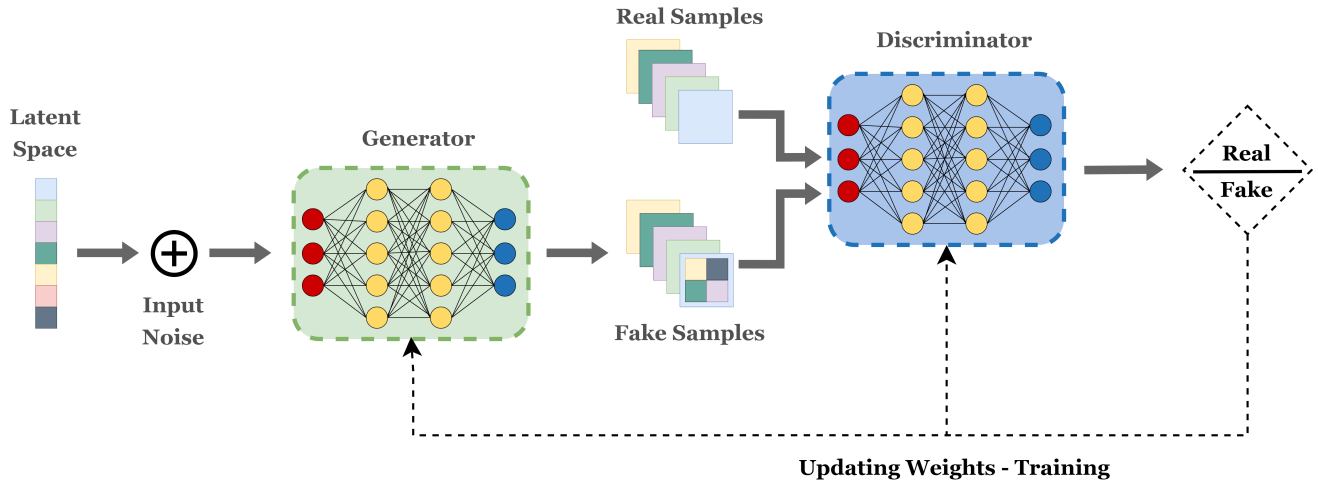


Figure 13. Generative Adversarial Network Architecture

Tabular GANs

Recently, GANs have been employed to effectively model tabular data with the goal of accurate data generation [85]. Generating tabular data has own challenges: Firstly, the mixture of various input data types (integers, decimals, categories, time, text, etc.). Secondly, the shapes of distribution (multi-modal, long tail, Non-Gaussian, etc.) can vary greatly depending on the input features. Lastly, the sparse one-hot-encoded vectors can lead to highly imbalanced categorical columns [86].

For our study, we utilized the Tabular-data-generation library [87], where we applied the two models: Tabular GAN (TGAN) and Conditional Tabular GAN (CTGAN), to synthetically generate tabular data.

- **Tabular GAN:** leverages deep neural networks to generate high-quality synthetic tables, managing both continuous and discrete variables, and scaling up for larger datasets through efficient inter-column correlations.
- **Conditional Tabular GAN:** extends traditional GANs by ensuring synthetic data aligns with the original data's conditional distributions. It leverages a generator and a discriminator, both conditioned on specific categorical variables, enabling the production of synthetic data that accurately represents the input data's conditional distributions. This makes it suitable for creating realistic synthetic data for various applications.

Methodology

Our experimental setup for testing our hypothesis is straightforward. We utilize our previously created dataset, preprocessed and split into training and testing sets. Then, we employ both TGAN and CTGAN models to produce two separate synthetic datasets. The experiment involves training various machine learning models on these synthetic datasets and the original data.

The goal is to compare the model's performance when trained on synthetic datasets against their performance on the original data, verified against the original data's test set. While we anticipate high accuracy scores from models trained on the original dataset, the focus of our study is to evaluate whether the models trained on synthetic datasets deliver comparable results.

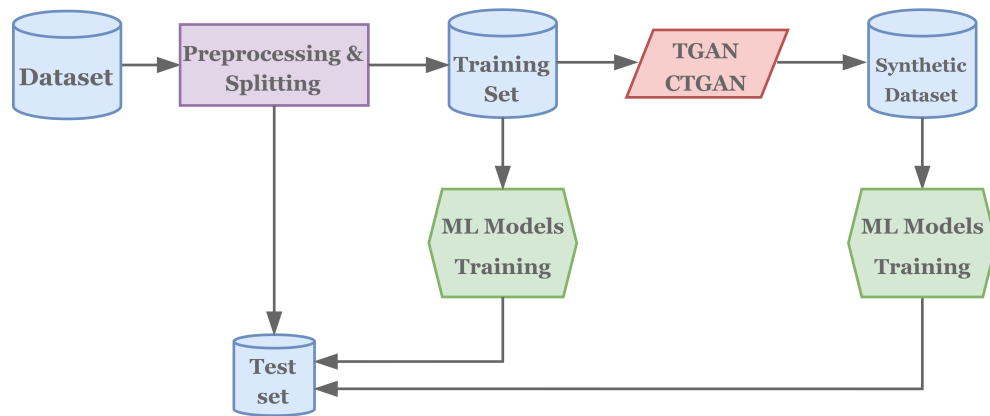


Figure 14. Experiment Methodology

5.3.3. Evaluation

We conduct comprehensive evaluations of the two key components in our study. First, we examine SecGPT, our interactive agent designed to enhance and automate the penetration testing process. Subsequently, we assess the synthetic datasets generated by the GAN pipeline, comparing their performance with the original data.

SecGPT Evaluation

The system is deployed as a terminal application to ensure compatibility with various environments, including nodes without a graphical user interface and those accessed remotely. As shown in Figure 15, the system can accurately retrieve content from the external knowledge base and integrate it correctly with its existing knowledge base. Additionally, it can execute commands on the local network to scan for available devices, as the initial step in our penetration testing methodology.

```

##### Welcome to SEC-GPT #####
#####
SEC-GPT
#####
your chosen model is: gpt-4o
Type 'exit' to end the conversation.

You: scan my network to identify the devices available
Assistant: I will use the arp-scan command to scan your
network and identify the devices available. Here is th
e command:

```shell
sudo arp-scan -l
```

This command will list all devices on your network along
with their IP and MAC addresses. Let me know if you n
eed any further assistance.
More Info. can be found here: scan-strategy.pdf
are you sure you want to run this code? (y/n): y

are you sure you want to run this code? (y/n): y

#####
Running Command Locally:
#####

[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 00:0c:29: ,
IPv4: 192.168.43.129
Starting arp-scan 1.10.0 with 256 hosts (https://github
.com/royhills/arp-scan)
192.168.43.1 00:50:56:  VMware, Inc.
192.168.43.2 00:50:56:  VMware, Inc.
192.168.43.136 00:0c:29:  VMware, Inc.
192.168.43.254 00:50:56:  VMware, Inc.

4 packets received by filter, 0 packets dropped by kern
el
Ending arp-scan 1.10.0: 256 hosts scanned in 2.012 seco
nds (127.24 hosts/sec). 4 responded

```

Figure 15. Sec-GPT User Interface

We evaluate the system using multiple attack scenarios listed in the knowledge base. For each attack, we conduct 10 trials. The outcome indicated by a check mark, represents the most occurring outcome across these trials.

- **Completed with No Guidance:** This denotes that the agent successfully identified the correct tool for the attack and implemented it locally with minimal user input (fewer than two assists after the initial attack choice inquiry).
- **Completed with User Help:** This indicates that the agent required more than two user interventions to complete the task successfully.
- **Failed:** This outcome means the agent either got stuck in an error-correcting loop or the attack was too complex to be executed by the agent within five replies.

Table 13. Attacks Performance

| Attack Scenario | Completed With no guidance | Completed with user help | Failed |
|-------------------------|----------------------------|--------------------------|--------|
| Portscan | ✓ | | |
| SlowHTTPTest | ✓ | | |
| Slowloris | ✓ | | |
| Dos Hulk | ✓ | | |
| Cross-Site-Scripting | | ✓ | |
| Infiltration - Backdoor | | | ✓ |
| Brute Force - FTP | ✓ | | |
| Brute Force - SSH | ✓ | | |

Synthetic Data Generation Evaluation

In order to validate the efficiency of the new dataset, a comparative analysis was carried out using it and the original dataset in an anomaly detection task, a typical machine learning task. A host of well-established classifiers such as Random Forest, k-Nearest Neighbors, Decision Tree (ID3), Gradient Boosting Machine, Multilayer Perceptron, and Quadratic Discriminant Analysis were evaluated in this study. The classifiers were

trained using the original dataset, followed by the evaluation of their accuracy on the original and the synthetic datasets created by TGAN and CTGAN.

The results summarized in Table.14 suggest that the artificially generated dataset gives predictive results comparable to the original dataset. Specifically, the TGAN-generated dataset performed better than the original in Quadratic Discriminant Analysis but reflects a slightly diminished result with the Multilayer Perceptron, and was generally similar or superior in performance to the CTGAN-generated dataset.

Table 14. Accuracy of ML models trained on real and synthetic training sets

| Algorithm | Real | TGAN | CTGAN |
|---------------------------------|-------|-------|-------|
| Random Forest | 0.979 | 0.972 | 0.974 |
| k-Nearest Neighbors | 0.970 | 0.924 | 0.897 |
| Decision Tree (ID3) | 0.978 | 0.969 | 0.969 |
| Gradient Boosting Machine | 0.979 | 0.973 | 0.972 |
| Multilayer Perceptron | 0.955 | 0.863 | 0.772 |
| Quadratic Discriminant Analysis | 0.899 | 0.924 | 0.931 |

A further analysis was undertaken to quantify various similarity metrics between the original dataset and those generated synthetically. These metrics encompassed several aspects including Uniqueness, Data Quality, and PSI similarity.

In this context, **Uniqueness** is defined as the proportion of non-null unique values present in both the generated and original datasets, taking into consideration the minimum number of rows.

Data Quality, on the other hand, is determined by the similarity in distribution between the two datasets, using the average score from a Kolmogorov-Smirnov test performed across the respective columns.

PSI Similarity involves computing the mean Population Stability Index (PSI) across all paired columns, although this is restricted to a theoretical maximum.

Total similarity score is calculated as an equation of these component scores, weighted accordingly: it constitutes 50% of the uniqueness score, 30% of the data quality score, and 20% of the PSI similarity score. To ensure validity, the final similarity score is constrained to lie between 0 and 1. The results are summarized in Table.15.

Table 15. Similarity analysis of real and synthetic training sets

| Metric | TGAN | CTGAN |
|------------------------|-------|-------|
| Uniqueness Score | 0.312 | 0.945 |
| Data Quality Score | 0.858 | 0.751 |
| PSI Similarity | 0.768 | 0.769 |
| Total Similarity Score | 0.567 | 0.852 |

6. DISCUSSION

This chapter provides an analysis of Large Language Models within the context of cyber defense operations, exploring their benefits and potential drawbacks. We will discuss the experimental challenges and limitations observed and outline future research directions aimed at optimizing the use of LLMs for enhanced cyber security.

6.1. Results

In this study, multiple experiments were conducted to evaluate the effectiveness of integrating LLMs into various cyber defense operations. Using several existing datasets and our own dataset—summarized in Table 3—we tested and compared their performance in machine learning-based intrusion detection systems classification tasks. The results, detailed in Table 4, validate the efficacy of our dataset, the reliability of our pipeline in data generation and real time detection, and demonstrate significant improvements in detection rates.

Regarding **HuntGPT**, The study illustrates the efficacy of integrating LLM-based conversational agents with Explainable AI within intrusion detection systems. Our prototype, HuntGPT, combines the advanced capabilities of GPT-3.5-turbo with a user-friendly dashboard and adeptly elucidates the latent details of detected anomalies. The model demonstrated substantial proficiency in cybersecurity knowledge, as evident from its success rates ranging between 72% and 82.5% on various reputable standardized certification exams in cybersecurity. However, these results also highlight areas for enhancement, mainly focusing on improving proficiency in fundamental cybersecurity concepts where the model achieved a lower success rate. The extensive readability analysis of generated responses indicates that the content produced is generally comprehensible for individuals with a basic college education level, fostering user understanding and interaction. The implemented conversational agent effectively generates actionable responses and promotes user engagement by communicating complex concepts, thereby making a substantial contribution to enhancing user comprehension in cybersecurity.

Moving on to **LLMs as Classifiers**, our results demonstrated in Tables 11 and 10 underscore the promising capabilities of fine-tuned language models such as BERT, RoBERTa, DeBERTa, and T5. These models, after being fine-tuned on network traffic data, exhibit significant potential in downstream tasks, effectively leveraging their advanced pattern recognition capabilities to excel in anomaly detection within cybersecurity operations.

Finally, for **Autonomous Data Regeneration**, we employed a hybrid approach that utilizes the organic data generation capabilities of SecGPT alongside a GAN architecture. This method synthesizes a comprehensive dataset by combining newly generated attack data with our original dataset, which contains a complete profile of manually executed attacks. The results of integrating these autonomously performed attacks in new network environments, as outlined in Table 14, demonstrate the

feasibility of creating a high-quality, synthetic dataset. This innovative approach has the potential to revolutionize IDS data generation by enabling continuous and autonomous execution and synthesis of multiple attacks across various network scenarios providing a robust foundation for retraining models and enhancing IDS performance and generalization capabilities.

6.2. Challenges

Our research is comprehensive, spanning various domains in cyber defense operations. It covers areas such as network security, automated network testing and vulnerability assessment, cyber threat awareness and education, along with data augmentation and synthesis. However, these efforts came with multiple challenges, including managing data volume in network traffic capture, the lack of reliable public datasets, adapting to dynamic and evolving cyber threats, and handling computationally intensive models.

6.2.1. Data Volume in Network Traffic Capture

Network traffic capture generates an overwhelming volume of data due to modern networks supporting an array of devices and services. The sheer quantity of packets exchanged can quickly escalate, creating a flood of data that traditional systems struggle to manage. This poses significant challenges in terms of storage, processing, and analysis, necessitating efficient strategies to handle the immense data flow effectively. To manage this, we integrated CICFlowMeter 2017, a tool that converts raw packets into condensed flow records. This transformation significantly reduces data volume, making storage and transmission more efficient. Additionally, it enhances analysis by aggregating essential information, improving overall system performance and response times to network threats.

6.2.2. Lack of Reliable Public Datasets

One major challenge was the scarcity of reliable public datasets for Intrusion Detection Systems. Many available datasets are outdated, lack diversity, and suffer from quality issues, making them unsuitable for current cyber threat landscapes. To address this, we created our own dataset. A lengthy process that involved: Data Collection, Annotation and Labeling, and Validation and Testing.

6.2.3. Adapting to Dynamic and Evolving Cyber Threats

The cyber threat landscape is continuously evolving, with new threats and variations of known attacks emerging constantly. This requires continuous updates and retraining of detection systems to maintain effectiveness. To address this, we integrated Large Language Models with a cloud-based knowledge base, allowing for real-time updates

on the latest threat intelligence. This ensures our system stays current and adaptable. Experiment 3 demonstrated the effectiveness of this approach. Key strategies included:

- **Continuous Data Integration:** Using cloud-based knowledge base to keep LLMs updated with the latest threats.
- **Comprehensive Attack Simulation:** Conducting a wide range of attack simulations with multiple tools to cover diverse threat scenarios in our dataset.
- **Synthetic Data Generation:** Utilizing GAN architecture to create synthetic data. This approach enhances our dataset and allows continuous learning and adaptation, making our IDS more robust and effective against emerging threats.

These strategies ensure our system remains resilient and capable of adapting to the rapidly changing cyber threat landscape.

6.2.4. Computationally Intensive Models

Training LLMs is an extremely hardware-intensive task, requiring significant computational resources. The complexity and size of these models necessitate powerful GPUs and substantial memory, which makes training from scratch both time-consuming and costly. Given these constraints, we opted for fine-tuning pre-trained models instead of training from scratch. Fine-tuning process starts with a model that has already been trained on a large corpus of text. The pre-trained model already understands general language patterns, so we only need to adjust its parameters to better fit our specific cyber defense context. This approach is considerably less resource-intensive and faster while still providing highly effective results.

6.3. Future Work

In our future work, we intend to progress in three strategic directions: Firstly, we aim to expand our dataset by incorporating a broader range of updated attack types and increasing the volume of data to address class imbalances more effectively. Secondly, we plan to enhance the capabilities of SecGPT, enabling it to autonomously execute a wider array of network attacks without requiring user intervention. This will further automate and refine our data generation process.

Concurrently, we are focused on embedding actionable AI capabilities within HuntGPT, enabling it to issue active commands directly to the Security Information and Event Management (SIEM) component, facilitating instantaneous and intelligent responses to security events

7. CONCLUSION

This research contributes valuable insights into integrating advanced AI models within interactive, user-focused applications in cybersecurity. The achieved success rates and the readability of generated responses by HuntGPT, the real time detection and performance of the designed pipeline, the high accuracy of Transformer models in anomaly detection, the efficacy of SecGPT in penetration testing automation and the quality of synthetic data generated emphasize the potential of implementing such integrated models in real-world applications, providing a solid foundation for developing more sophisticated, explainable, interpretable, actionable, and user-friendly cybersecurity solutions.

Our research has demonstrated that LLMs possess the capability to extend beyond traditional natural language processing tasks and seamlessly integrate with various traditional AI methodologies. We successfully incorporated LLMs to expand existing explainability frameworks, offering a novel approach that can be applied to domains beyond cybersecurity. Additionally, we combined these models with conventional deep learning techniques, such as those used in GANs, to significantly enhance the quality of synthetic data output. This integration underscores the exceptional value LLMs bring to the development of advanced and comprehensive next-generation cybersecurity applications.

This study serves as a stepping stone for further research and development in creating LLM-driven security tools that integrate XAI, Deep Learning and Machine Learning in response to the evolving landscape of cybersecurity threats and user needs.

8. REFERENCES

- [1] Linkov I., Galaitsi S., Trump B.D., Keisler J.M. & Kott A. (2020) Cybertrust: From explainable to actionable and interpretable artificial intelligence. *Computer* 53, pp. 91–96.
- [2] Morgan S., 2023 Cybersecurity Almanac: 100 Facts, Figures, Predictions, And Statistics — cybersecurityventures.com. <https://cybersecurityventures.com/cybersecurity-almanac-2023/>. [Accessed 23-09-2023].
- [3] Cybersecurity C.I. (2018) Framework for improving critical infrastructure cybersecurity. URL: <https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.4162018>.
- [4] Shu X., Araujo F., Schales D.L., Stoecklin M.P., Jang J., Huang H. & Rao J.R. (2018) Threat intelligence computing. In: *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, pp. 1883–1898.
- [5] Bhuyan M.H., Bhattacharyya D.K. & Kalita J.K. (2014) Network anomaly detection: Methods, systems and tools. *IEEE Communications Surveys & Tutorials* 16, pp. 303–336.
- [6] Mavroeidis V. & Jøsang A. (2018) Data-driven threat hunting using sysmon. In: *Proceedings of the 2nd International Conference on Cryptography, Security and Privacy, ICCSP 2018, Association for Computing Machinery, New York, NY, USA*, p. 82–88. URL: <https://doi.org/10.1145/3199478.3199490>.
- [7] Subramanian K. & Meng W. (2021) Threat hunting using elastic stack: An evaluation. In: *2021 IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI)*, pp. 1–6.
- [8] Aldauji F., Batarfi O. & Bayousef M. (2022) Utilizing cyber threat hunting techniques to find ransomware attacks: A survey of the state of the art. *IEEE Access* 10, pp. 61695–61706.
- [9] Thottan M. & Ji C. (2003) Anomaly detection in ip networks. *IEEE Transactions on Signal Processing* 51, pp. 2191–2204.
- [10] Cavallaro L., Kinder J., Pendlebury F. & Pierazzi F. (2023) Are machine learning models for malware detection ready for prime time? *IEEE Security & Privacy* 21, pp. 53–56.
- [11] Charmet F., Tanuwidjaja H.C., Ayoubi S., Gimenez P.F., Han Y., Jmila H., Blanc G., Takahashi T. & Zhang Z. (2022) Explainable artificial intelligence for cybersecurity: a literature survey. *Annals of Telecommunications* 77, pp. 789–812.

- [12] Nyre-Yu M., Morris E., Moss B.C., Smutz C. & Smith M. (2022) Explainable ai in cybersecurity operations: Lessons learned from xai tool deployment. In: Proceedings of the Usable Security and Privacy (USEC) Symposium, San Diego, CA, USA, vol. 28, vol. 28.
- [13] Nguyen T., Nguyen H., Ijaz A., Sheikhi S., Vasilakos A.V. & Kostakos P. (2024) Large language models in 6g security: challenges and opportunities. arXiv preprint arXiv:2403.12239 .
- [14] Balasubramanian P., Seby J. & Kostakos P. (2023) Transformer-based llms in cybersecurity: An in-depth study on log anomaly detection and conversational defense mechanisms. In: 2023 IEEE International Conference on Big Data (BigData), IEEE, pp. 3590–3599.
- [15] Kaheh M., Kholgh D.K. & Kostakos P. (2023) Cyber sentinel: Exploring conversational agents in streamlining security tasks with gpt-4. arXiv preprint arXiv:2309.16422 .
- [16] Ali T. & Kostakos P. (2023) Huntgpt: Integrating machine learning-based anomaly detection and explainable ai with large language models (llms). arXiv preprint arXiv:2309.16021 .
- [17] Gregory P. (2023) CISM Certified Information Security Manager Practice Exams, Second Edition. McGraw Hill LLC. URL: <https://books.google.fi/books?id=4I-nEAAAQBAJ>.
- [18] Heikkilä M., Rättyä A., Pieskä S. & Jämsä J. (2016) Security challenges in small- and medium-sized manufacturing enterprises. In: 2016 International Symposium on Small-scale Intelligent Manufacturing Systems (SIMS), pp. 25–30.
- [19] Inc S. (2022), What is the cost of building a robust 24/7 soc for your organization. URL: <https://sharkstriker.com/blog/what-is-the-cost-of-building-a-robust-24-7-soc-for-your-organization/>.
- [20] Hassanin M. & Moustafa N. (2024) A comprehensive overview of large language models (llms) for cyber defences: Opportunities and directions. arXiv preprint arXiv:2405.14487 .
- [21] Bhattacharyya D.K. & Kalita J. (2013) Network Anomaly Detection: A Machine Learning Perspective.
- [22] Mahmoodi A.B.Z., Sheikhi S., Peltonen E. & Kostakos P. (2023) Autonomous federated learning for distributed intrusion detection systems in public networks. IEEE Access 11, pp. 121325–121339.
- [23] Sheikhi S. & Kostakos P. (2023) Cyber threat hunting using unsupervised federated learning and adversary emulation. In: 2023 IEEE International Conference on Cyber Security and Resilience (CSR), IEEE, pp. 315–320.

- [24] Sheikhi S. & Kostakos P. (2023) Ddos attack detection using unsupervised federated learning for 5g networks and beyond. In: 2023 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit), pp. 442–447.
- [25] Yihunie F., Abdelfattah E. & Regmi A. (2019) Applying machine learning to anomaly-based intrusion detection systems. In: 2019 IEEE Long Island Systems, Applications and Technology Conference (LISAT), pp. 1–5.
- [26] Eltanbouly S., Bashendy M., AlNaimi N., Chkirbene Z. & Erbad A. (2020) Machine learning techniques for network anomaly detection: A survey. In: 2020 IEEE International Conference on Informatics, IoT, and Enabling Technologies (ICIoT), pp. 156–162.
- [27] Zhao Y., Chen J., Wu D., Teng J. & Yu S. (2019) Multi-task network anomaly detection using federated learning. In: Proceedings of the 10th International Symposium on Information and Communication Technology, Association for Computing Machinery, p. 273–279. URL: <https://doi.org/10.1145/3368926.3369705>.
- [28] Preuveneers D., Rimmer V., Tsingenopoulos I., Spooren J., Joosen W. & Ilie-Zudor E. (2018) Chained anomaly detection models for federated learning: An intrusion detection case study. Applied Sciences 8. URL: <https://www.mdpi.com/2076-3417/8/12/2663>.
- [29] Mothukuri V., Khare P., Parizi R.M., Pouriyeh S., Dehghantanha A. & Srivastava G. (2022) Federated-learning-based anomaly detection for iot security attacks. IEEE Internet of Things Journal 9, pp. 2545–2554.
- [30] Nguyen T.D., Marchal S., Miettinen M., Fereidooni H., Asokan N. & Sadeghi A.R. (2019) DIot: A federated self-learning anomaly detection system for iot. In: 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS), pp. 756–767.
- [31] Porambage P., Gür G., Osorio D.P.M., Liyanage M., Gurtov A. & Ylianttila M. (2021) The roadmap to 6g security and privacy. IEEE Open Journal of the Communications Society 2, pp. 1094–1122.
- [32] Gunning D., Stefik M., Choi J., Miller T., Stumpf S. & Yang G.Z. (2019) Xai—explainable artificial intelligence. Science Robotics 4, p. eaay7120.
- [33] Nguyen Q.P., Wai L., Divakaran D.M., Low K. & Chan M. (2019) Gee: A gradient-based explainable variational autoencoder for network anomaly detection.
- [34] Han D., Wang Z., Chen W., Zhong Y., Wang S., Zhang H., Yang J., Shi X. & Yin X. (2021), Deepaid: Interpreting and improving deep learning-based anomaly detection in security applications.

- [35] Nadeem A., Vos D., Cao C., Pajola L., Dieck S., Baumgartner R. & Verwer S. (2023) Sok: Explainable machine learning for computer security applications. In: 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P), IEEE, pp. 221–240.
- [36] Peretz D.B. (2020), A siri for network security: How chatbots can enhance business agility. URL: <https://www.infosecurity-magazine.com/opinions/network-chatbots-agility/>.
- [37] Franco M.F., Rodrigues B., Scheid E.J., Jacobs A., Killer C., Granville L.Z. & Stiller B. (2020) Secbot: a business-driven conversational agent for cybersecurity planning and management. In: 2020 16th International Conference on Network and Service Management (CNSM), pp. 1–7.
- [38] McIntosh T., Liu T., Susnjak T., Alavizadeh H., Ng A., Nowrozy R. & Watters P. (2023) Harnessing gpt-4 for generation of cybersecurity grc policies: A focus on ransomware attack mitigation. *Computers & Security* 134.
- [39] Setianto F., Tsani E., Sadiq F., Domalis G., Tsakalidis D. & Kostakos P. (2022) Gpt-2c: A parser for honeypot logs using large pre-trained language models. In: Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, ASONAM '21, Association for Computing Machinery, New York, NY, USA, p. 649–653. URL: <https://doi.org/10.1145/3487351.3492723>.
- [40] Nour B., Pourzandi M. & Debbabi M. (2023) A survey on threat hunting in enterprise networks. *IEEE Communications Surveys & Tutorials* .
- [41] Sebastian G. (2023) Do chatgpt and other ai chatbots pose a cybersecurity risk?: An exploratory study. *International Journal of Security and Privacy in Pervasive Computing (IJSPPC)* 15, pp. 1–11.
- [42] Yue T. (2023), Democratizing financial knowledge with chatgpt by openai: Unleashing the power of technology.
- [43] Wikipedia contributors (2023), Prompt engineering. URL: https://en.wikipedia.org/w/index.php?title=Prompt_engineering&oldid=1176364192, [Online; accessed 26-September-2023].
- [44] Si C., Gan Z., Yang Z., Wang S., Wang J., Boyd-Graber J. & Wang L. (2022), Prompting gpt-3 to be reliable.
- [45] Gilson A., Safranek C.W., Huang T., Socrates V., Chi L., Taylor R.A. & Chartash D. (2023) How does chatgpt perform on the united states medical licensing examination? the implications of large language models for medical education and knowledge assessment. *JMIR Med Educ* 9, p. e45312.
- [46] Chu M.N. (2023) Assessing the benefits of chatgpt for business: An empirical study on organizational performance. *IEEE Access* 11, pp. 76427–76436.

- [47] Gniewkowski M., Maciejewski H., Surmacz T. & Walentynowicz W. (2023) Sec2vec: Anomaly detection in http traffic and malicious urls. In: Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing, pp. 1154–1162.
- [48] John A. et al. (2022) A deep learning based intrusion detection system using transformers .
- [49] Deng G., Liu Y., Mayoral-Vilches V., Liu P., Li Y., Xu Y., Zhang T., Liu Y., Pinzger M. & Rass S. (2023) Pentestgpt: An llm-empowered automatic penetration testing tool. arXiv preprint arXiv:2308.06782 .
- [50] Moskal S., Laney S., Hemberg E. & O'Reilly U.M. (2023) Llms killed the script kiddie: How agents supported by large language models change the landscape of network threat testing. arXiv preprint arXiv:2310.06936 .
- [51] Fang R., Bindu R., Gupta A., Zhan Q. & Kang D. (2024) Llm agents can autonomously hack websites. arXiv preprint arXiv:2402.06664 .
- [52] Neelakantan S. & Rao S. (2012) A threat-aware hybrid intrusion–detection architecture for dynamic network environments. *CSI J. Comput* 1.
- [53] Ghafouri A., Abbas W., Laszka A., Vorobeychik Y. & Koutsoukos X. (2016) Optimal thresholds for anomaly-based intrusion detection in dynamical environments. In: *Decision and Game Theory for Security: 7th International Conference, GameSec 2016, New York, NY, USA, November 2-4, 2016, Proceedings 7*, Springer, pp. 415–434.
- [54] Pietraszek T. & Tanner A. (2005) Data mining and machine learning—towards reducing false positives in intrusion detection. *Information security technical report* 10, pp. 169–183.
- [55] Sharafaldin I., Lashkari A.H. & Ghorbani A.A. (2018) Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 1, pp. 108–116.
- [56] Garcia-Teodoro P., Diaz-Verdejo J., Maciá-Fernández G. & Vázquez E. (2009) Anomaly-based network intrusion detection: Techniques, systems and challenges. *computers & security* 28, pp. 18–28.
- [57] Shiravi A., Shiravi H., Tavallaei M. & Ghorbani A.A. (2012) Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *computers & security* 31, pp. 357–374.
- [58] Thakkar A. & Lohiya R. (2020) A review of the advancement in intrusion detection datasets. *Procedia Computer Science* 167, pp. 636–645.
- [59] Sommer R. & Paxson V. (2010) Outside the closed world: On using machine learning for network intrusion detection. In: *2010 IEEE Symposium on Security and Privacy*, pp. 305–316.

- [60] Kenyon A., Deka L. & Elizondo D. (2020) Are public intrusion datasets fit for purpose characterising the state of the art in intrusion event datasets. *Computers & Security* 99, p. 102022.
- [61] Thomas C., Sharma V. & Balakrishnan N. (2008) Usefulness of darpa dataset for intrusion detection system evaluation. In: *Data Mining, Intrusion Detection, Information Assurance, and Data Networks Security 2008*, vol. 6973, SPIE, vol. 6973, pp. 164–171.
- [62] Olusola A.A., Oladele A.S. & Abosede D.O. (2010) Analysis of kdd'99 intrusion detection dataset for selection of relevance features. In: *Proceedings of the world congress on engineering and computer science*, vol. 1, WCECS, vol. 1, pp. 20–22.
- [63] Tavallaee M., Bagheri E., Lu W. & Ghorbani A.A. (2009) A detailed analysis of the kdd cup 99 data set. In: *2009 IEEE symposium on computational intelligence for security and defense applications*, Ieee, pp. 1–6.
- [64] Def con capture the flag (ctf) archive. URL: <https://defcon.org/html/links/dc-ctf.html>.
- [65] Shirsath V. (2023), Caida ucsd ddos 2007 attack dataset. URL: <https://dx.doi.org/10.21227/dvp9-s124>.
- [66] Nechaev B., Allman M., Paxson V. & Gurtov A.V. (2010) A preliminary analysis of tcp performance in an enterprise network. *INM/WREN* 10.
- [67] Sangster B., O'connor T., Cook T., Fanelli R., Dean E., Morrell C. & Conti G.J. (2009) Toward instrumenting network warfare competitions to generate labeled datasets. In: *CSET*.
- [68] Song J., Takakura H., Okabe Y., Eto M., Inoue D. & Nakao K. (2011) Statistical analysis of honeypot data and building of kyoto 2006+ dataset for nids evaluation. In: *Proceedings of the first workshop on building analysis datasets and gathering experience returns for security*, pp. 29–36.
- [69] Sperotto A., Sadre R., Van Vliet F. & Pras A. (2009) A labeled data set for flow-based intrusion detection. In: *IP Operations and Management: 9th IEEE International Workshop, IPOM 2009, Venice, Italy, October 29-30, 2009. Proceedings* 9, Springer, pp. 39–50.
- [70] Kenyon A., Deka L. & Elizondo D. (2020) Are public intrusion datasets fit for purpose characterising the state of the art in intrusion event datasets. *Computers & Security* 99, p. 102022.
- [71] Creech G. (2014) Developing a high-accuracy cross platform host-based intrusion detection system capable of reliably detecting zero-day attacks. Ph.D. thesis, UNSW Sydney.
- [72] Borisaniya B. & Patel D. (2015) Evaluation of modified vector space representation using adfa-ld and adfa-wd datasets. *Journal of Information Security* 6, pp. 250–264.

- [73] Panigrahi R. & Borah S. (2018) A detailed analysis of cicids2017 dataset for designing intrusion detection systems. *International Journal of Engineering & Technology* 7, pp. 479–482.
- [74] s0md3v (Accessed: 04.05.2024), XSSStrike: Most advanced XSS detection suite. <https://github.com/s0md3v/XSSStrike>.
- [75] Wikipedia contributors (2023), Isaca. URL: <https://en.wikipedia.org/w/index.php?title=ISACA&oldid=1170788297>, [Online; accessed 3-August-2023].
- [76] Jain A. (2023), Cism difficulty level: Exam format and study guide. URL: <https://www.knowledgehut.com/blog/security/cism-difficulty-level#cism-exam-format>.
- [77] py-readability-metrics: A Python library for computing readability metrics. <https://pypi.org/project/py-readability-metrics/>. Accessed: Sep. 2023.
- [78] Devlin J., Chang M.W., Lee K. & Toutanova K. (2018) Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- [79] Liu Y., Ott M., Goyal N., Du J., Joshi M., Chen D., Levy O., Lewis M., Zettlemoyer L. & Stoyanov V. (2019) Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.
- [80] He P., Liu X., Gao J. & Chen W. (2020) Deberta: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*.
- [81] Raffel C., Shazeer N., Roberts A., Lee K., Narang S., Matena M., Zhou Y., Li W. & Liu P.J. (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, pp. 1–67.
- [82] Zhang J. & Zulkernine M. (2006) Anomaly based network intrusion detection with unsupervised outlier detection. In: 2006 IEEE International Conference on Communications, vol. 5, IEEE, vol. 5, pp. 2388–2393.
- [83] (2024), Microsoft azure. URL: <https://azure.microsoft.com/en-us/>, accessed: 2024-05-07.
- [84] Goodfellow I., Pouget-Abadie J., Mirza M., Xu B., Warde-Farley D., Ozair S., Courville A. & Bengio Y. (2020) Generative adversarial networks. *Communications of the ACM* 63, pp. 139–144.
- [85] Kholgh D.K. & Kostakos P. (2023) Pac-gpt: A novel approach to generating synthetic network traffic with gpt-3. *IEEE Access*.
- [86] Ganji D. & Chakrabortii C. (2023) Towards data generation to alleviate privacy concerns for cybersecurity applications. In: 2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC), IEEE, pp. 1447–1452.
- [87] Ashrapov I. (2020), Tabular gans for uneven distribution.